INTROL: Einführendes Labor Sicherheit

Kursmaterial HS25 - 2025

Florian Wamser & Ron Porath, Hochschule Luzern

8. Oktober 2025

INTROL: Einführendes Labor Sicherheit

Letzte Änderung: 8. Oktober 2025

Begleitendes Kursmaterial des Moduls *INTROL* im Herbstsemester 2025 an der Hochschule Luzern. Kursmaterial für einen Semesterkurs mit 14 Wochenstunden.

⚠ *Dieses Skript wird während der Vorlesung angepasst und vervollständigt. Neue Kapitel und Übungen kommen im Verlauf der Vorlesung hinzu.*

Autoren

Thomas Jösler: Initiale Version
S. Boussaha, N. Neher: Anpassungen 2019
Raphael Bolfing: Überarbeitung
Christian Scherrer, Nicolas Richner: Überarbeitungen
Christian Scherrer: Überarbeitung und Anpassungen bei webauthn.io GET/POST
Dr. Florian Wamser, florian.wamser@hslu.ch
Dr. Ron Porath, ron.porath@hslu.ch

Ideen zum Inhalt und initiales Feedback: Thomas Jösler, Peter Infanger

Ausgabe 2025

Erstmalige Durchführung im H21 (2021).

Studentische Angaben

Bitte tragen Sie im folgenden Feld ihren Namen und Informationen zu ihrem Team ein, falls Sie in einem Team arbeiten.

Team: *

Vorname 1: *

Nachname 1: *

Vorname 2: *

Nachname 2: *

Vorname 3: *

Nachname 3: *

* Je nach Teamgrösse müssen die Felder *Vor-* und *Nachname* in der entsprechenden Anzahl ausgefüllt werden.

Sonstige Angaben und Anmerkungen:

Inhaltsverzeichnis

1 Vorwort **3**
1.1 Feedback . . . 3
1.2 Legende . . . 3

2 Labor 3: Web Authentication **5**
2.1 Vorbereitung der Labor-Infrastruktur . . . 6
2.1.1 IIS beenden (nur nötig bei Fehlermeldung beim Starten der node.js-Anwendung) . . . 7

3 Cookie-based Authentication: Zustandsbehaftete Web-Authentifizierung mit Cookies **11**
3.1 Applikation öffnen . . . 11
3.2 Login durchführen . . . 13
3.3 Analyse . . . 15

4 Open Authorization (OAuth 2.0) mit OpenID Connect (OIDC): Sichere API-Autorisierung für Desktop-, Web- und Mobile-Anwendungen **21**
4.1 Starten der Applikationen . . . 21
4.2 Konfiguration des Zed Attack Proxys (ZAP) mit Mozilla Firefox . . . 22
4.3 Analyse des Ablaufs von OAuth 2.0 . . . 26
4.3.1 Schritt ①: Click login link . . . 27
4.3.2 Schritt ②: Authorization Code Request to /authorize . . . 28
4.3.3 Schritt ③: Redirect to login/authorization prompt . . . 30
4.3.4 Schritt ④: Authenticate and Consent . . . 31
4.3.5 Schritt ⑤: Authorization Code . . . 31
4.3.6 Schritt ⑥: Authorization Code + Client ID + Client Secret to /token . . . 31
4.3.7 Schritt ⑦: Validate Authorization Code + Client ID + Client Secret . . . 32
4.3.8 Schritt ⑧: ID Token and Access Token . . . 34
4.3.9 Schritt ⑨: Request user data with Access Token . . . 37
4.3.10 Schritt ⑩: Response . . . 37
4.4 Abschluss OAuth 2.0 . . . 39

5 WebAuthn: Programmierschnittstelle für eine direkte Authentifikation mittels Public Key-Verfahren **43**
5.1 Konfiguration von OWASP ZAP mit Microsoft Edge . . . 46
5.2 Konfiguration TPM & Windows Hello . . . 48

5.3 Kennenlernen des WebAuthn-Mechanismus 50
5.3.1 `Sign` Up: Erstellen eines Bentzerkontos bei der Webanwendung (Registrierung) 51
5.3.2 Login-Prozess 61
5.4 Abschluss WebAuthn 69

6 Aufräumen **71**

1 Vorwort

1.1 Feedback

> *Mit Ihrer Mithilfe kann die Qualität des Kurses laufend den Bedürfnissen angepasst und verbessert werden.*

Sollte etwas in diesem Kurs nicht wie beschrieben funktionieren, melden Sie dies bitte direkt dem Lehrpersonal oder per E-Mail an Florian Wamser (florian.wamser@hslu.ch).

Bei Problemen wenden Sie sich bitte ebenfalls an das Lehrpersonal.

1.2 Legende

Wichtige Begriffe und Schlüsselwörter sind *kursiv* gedruckt.
Auffällige Unteraussagen im Text sind **fett** gedruckt.

Kommandozeilenbefehle werden durch separate Felder mit grauem Hintergrund hervorgehoben. Zu Beginn einer Zeile sind Kommandozeilenbefehle mit einem > oder $ gekennzeichnet. Für Referenzierungen wird bei grösseren Blöcken eine Zeilennummer angegeben. Weder das Kommandozeilenzeichen noch die Zeilennummer ist Teil der Eingabe.

```
1 echo "Hello World!"
2 exit
```

Skriptinhalte oder Inhalte von Konfigurationsdateien werden ebenfalls in separaten Feldern mit grauem Hintergrund angegeben.

```
    #!/bin/bash
    echo "Hello, world!"
    echo the $# parameter did not destroy pygments syntax highlighting
    time find . -type f -exec ls -l {} \; | awk '{sum += $5} END {print sum
       }'
```

Anmerkungen und Informationen sind weiterführende Hinweise zu Befehlen oder zu der aktuellen Aufgabe. Sie gelten als ergänzender Hinweis, Notiz oder Hilfestellung.

Dringend beachten. Was hier steht, unbedingt merken oder ausführen. Oft müssen Befehle zwingend für den folgenden Aufgabenteil ausgeführt werden.

Weiterführende Literaturhinweise oder Links. Dies sind Informationen, die nicht zur Ausführung der Versuche benötigt werden, aber bekannt sein sollten.

2 Labor 3: Web Authentication

Zielsetzung

- Erlernen der Grundlagen zu Web-Authentifizierung, einschliesslich Challenge Response-Verfahren und Public Key-Infrastruktur (PKI)
- Studieren von Authentifizierungsmechanismen für Websites und webbasierte Dienste
  1. Cookie-basierte Authentifizierung
  2. Open Authorization (OAuth 2.0): sichere API-Autorisierung für Desktop-, Web- und Mobile-Anwendungen
  3. WebAuthn: Programmierschnittstelle, mit der Webanwendungen und Websites ihren Benutzern eine direkte Authentifikation mittels Public Key-Verfahren im Webbrowser anbieten können.
- Schrittweises Kennenlernen der Funktionsweise & des technischen Ablaufs jeder Lösung
- Sicherheitsaspekte jeder Lösung
- Verwendung des Trusted Platform Module (TPM) mit WebAuthn

Diese Laborübung soll Ihnen die Funktionsweise und den technischen Ablauf der derzeit gängigsten Web-Authentifizierungsverfahren zeigen. Sie erfahren, welche der verschiedenen Authentifizierungsmechanismen welche Vor- und Nachteile haben.

Zunächst wird die Cookie-basierte Authentifizierung und ihre Funktionsweise vorgestellt. Anschliessend wird gezeigt, wie man mittels OpenID Connect mit Anmeldedaten eines Drittanbieters die Anmeldung auf der eigenen Website verwalten kann. Am Ende wird per WebAuthn gezeigt, wie man das Trusted Platform Module (TPM) einbinden und nutzen kann.

Benötigte Mittel / Materialien

Verwenden Sie für diesen Versuch die Labor-Arbeitsstationen. Jedes Labor-Team benötigt einen Labor-Doppelarbeitsplatz. Gesamthaft wird folgende Hard- und Software benötigt:

1x Labor PC, mindestens Windows 10 1809 mit Microsoft Edge und folgender Software:

- Google Chrome
- Firefox
- ZAP Proxy
- Node.js

2.1 Vorbereitung der Labor-Infrastruktur

Dieses Kapitel beinhaltet alle Aufgaben, die ausgeführt werden müssen, bevor mit der eigentlichen Übung begonnen werden kann.

Der in den ersten Teilen der Übung verwendete Webbrowser ist `Google Chrome`. Natürlich könnten Sie auch einen Browser Ihrer Wahl verwenden. Unterstützung erhalten Sie hier jedoch nur, wenn Sie den jeweils angegebenen Browser verwenden.
Im dritten Teil `WebAuthn` muss zwingend `Microsoft Edge` verwendet werden. Andere Browser unterstützen zum heutigen Zeitpunkt die Integration des `TPM`-Moduls noch nicht.

1. Starten Sie die erste Applikation von der Kommandozeile aus:

```
C:\Windows\system32> cd \Users\cisco\Documents\web-
    authentication\cookie-authentication
C:\Users\cisco\Documents\web-authentication\cookie-
    authentication> npm start
```

Es ist zwingend notwendig, die Anwendungen in dem Ordner zu starten, in dem alle notwendigen Dateien und Daten liegen. Daher ist der Wechsel in den Ordner `web-authentication\cookie-authentication` zwingend notwendig. Dies gilt auch für alle weiteren `node.js`-Anwendungen, die noch im folgenden Verlauf genutzt werden.

Sollten Sie beim Starten der `Authentication App` auf der Kommandozeile die folgende Fehlermeldung erhalten, dann läuft auf Ihrer Workstation möglicherweise der `IIS-Webserver` auf diesem Port. Die `App` kann auf Port 80 nicht gestartet werden, weil der Port schon in Benutzung ist.

```
> Error: listen EACCES: permission denied
    0.0.0.0:80`), dann läuft auf Ihrer Workstation
    schon der IIS Webserver auf diesem Port.
```

Wenn dies der Fall ist, und nur dann, fahren Sie beim folgenden Kapitel 2.1.1 fort. Andernfalls können Sie direkt mit Kapitel 3 beginnen..

2.1.1 IIS beenden (nur nötig bei Fehlermeldung beim Starten der `node.js`-Anwendung)

1. Öffnen Sie auf Ihrer Windows-Workstation die `Internet Information Services (IIS)Management Console`. Ziel ist es den ISS zu stoppen oder zu deaktivieren, damit der Port nicht mehr belegt ist und wir unseren eigenen `node.js`-Webserver starten können.

Sollte auf Ihrem PC die `Internet Information Services (IIS)Management Console` nicht vorhanden sein, können Sie sie unter `Windows Features, Internet Information Services\Web Management Tools\IIS Management Console` aktivieren.

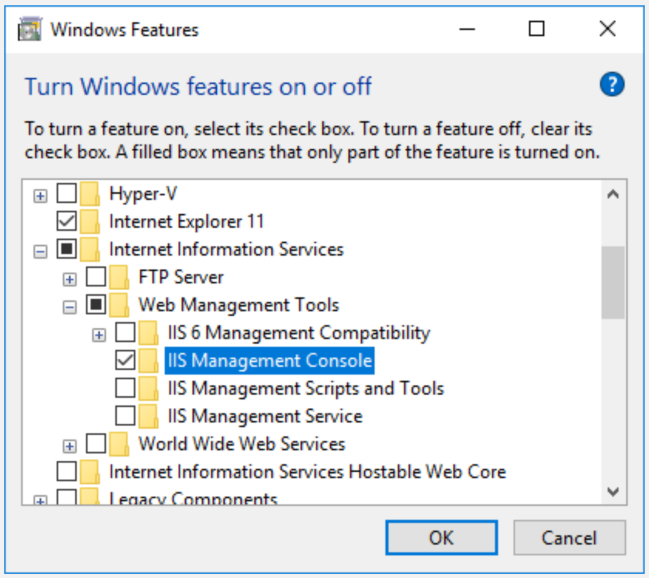


Abbildung 1: `Internet Information Services (IIS) Management Console` aktivieren

2. Falls neben dem `ISS Manager` der `Internet Information Services (IIS) -Dienst` installiert ist, sehen Sie oben rechts im Feld `Actions` den Punkt `Manage Server`. Klicken Sie hier auf `Stop`.

Beachten Sie, dass das Feld `Actions` nur erscheint, wenn der `Internet Information Services (IIS)-Dienst` installiert ist.

Da wir den Dienst deaktieren möchten und *nicht* nutzen werden, sollte im Normalfall (a) *keine* Fehlermeldung beim Starten der `node.js`-Anwendung kommen, siehe Kapitel 2.1, und (b) der IIS-Dienst gar nicht installiert oder gestartet sein.

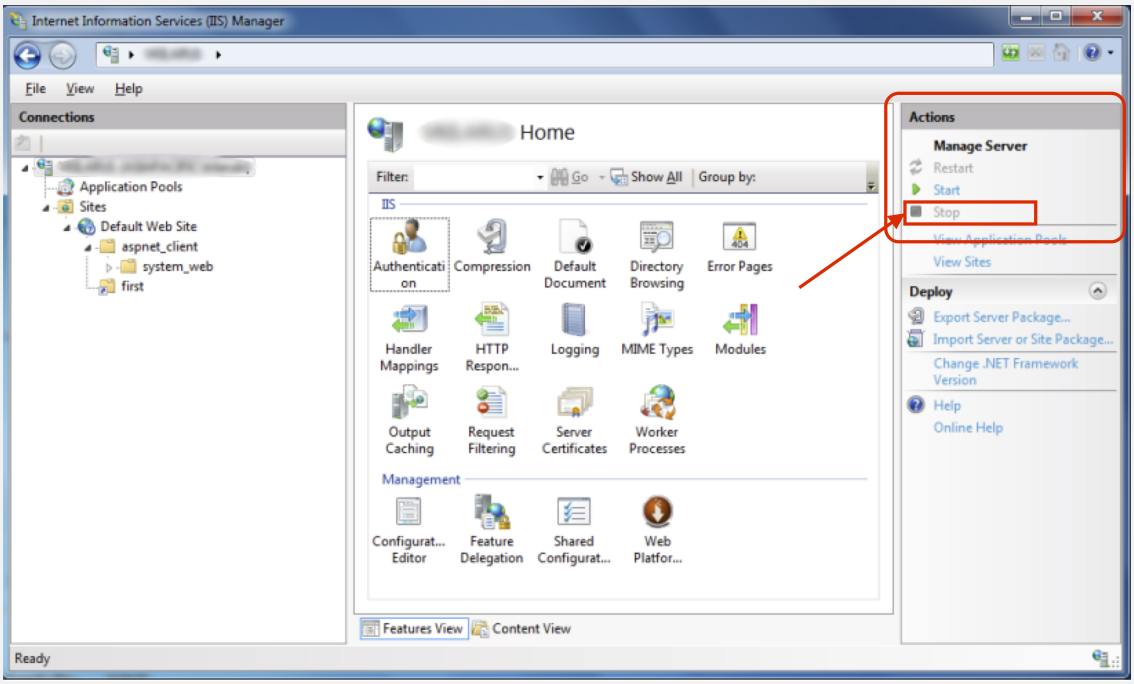


Abbildung 2: Beenden des `Internet Information Services (IIS)-Dienst` in der `IIS Management Console`.

3. Starten Sie die `Authentication Application` erneut:

```
C:\Windows\system32> cd \Users\cisco\Documents\web-
    authentication\cookie-authentication
C:\Users\cisco\Documents\web-authentication\cookie-
    authentication> npm start
```

Bitte überprüfen Sie an *allen* Arbeitsplätzen sorgfältig, ob alte Einstellungen von vorherigen Übungen, wie beispielsweise der Proxy-Server unter Windows oder die Proxy-Einstellungen in den Browsern `Firefox` oder `Chrome`, *nicht* aktiviert sind. Es kommt häufig vor, dass Studenten die Einstellungen unter Windows vergessen zurückzusetzen.

1. Öffnen Sie die Windows-Einstellungen, indem Sie die Tastenkombination `[Windows] + [I]` verwenden. Dort klicken Sie dann auf `Netzwerk & internet`.
2. Wechseln Sie dann in die Kategorie `Proxy`. Im rechten Teil des Fensters deaktivieren Sie nun die Option `Einstellungen automatisch erkennen` oder `Automatically detect settings`.
3. Unter dem Punkt `Manual proxy setup` deaktivieren Sie die Option `Use a proxy server: Off`.
4. Wechseln Sie in den `Mozilla Firefox`: `Settings` → `General` → `Network Settings`. Setzen Sie `No proxy`.

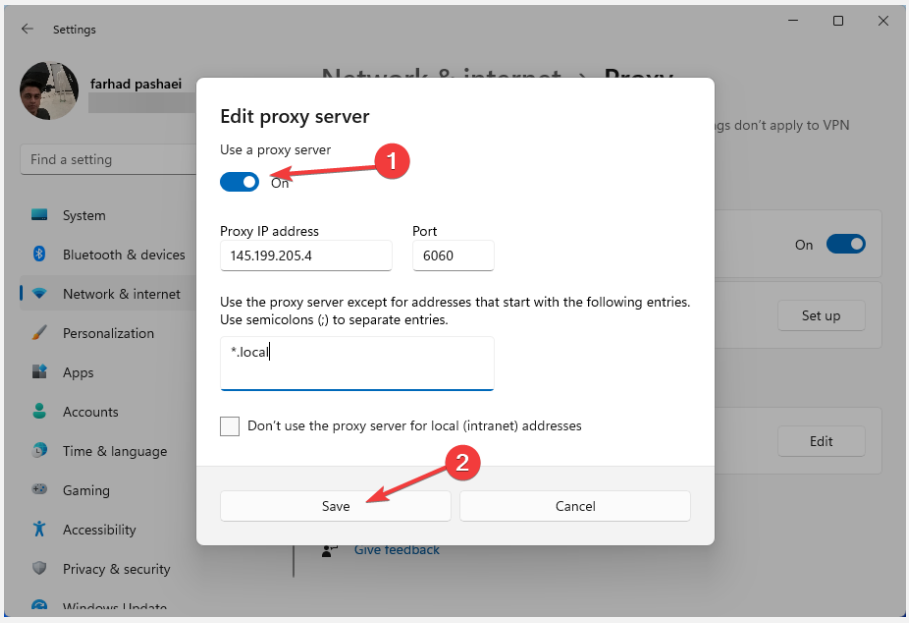


Abbildung 3: Deaktivieren der systemweiten Proxy-Einstellung unter Windows: `Use a proxy server: Off`.

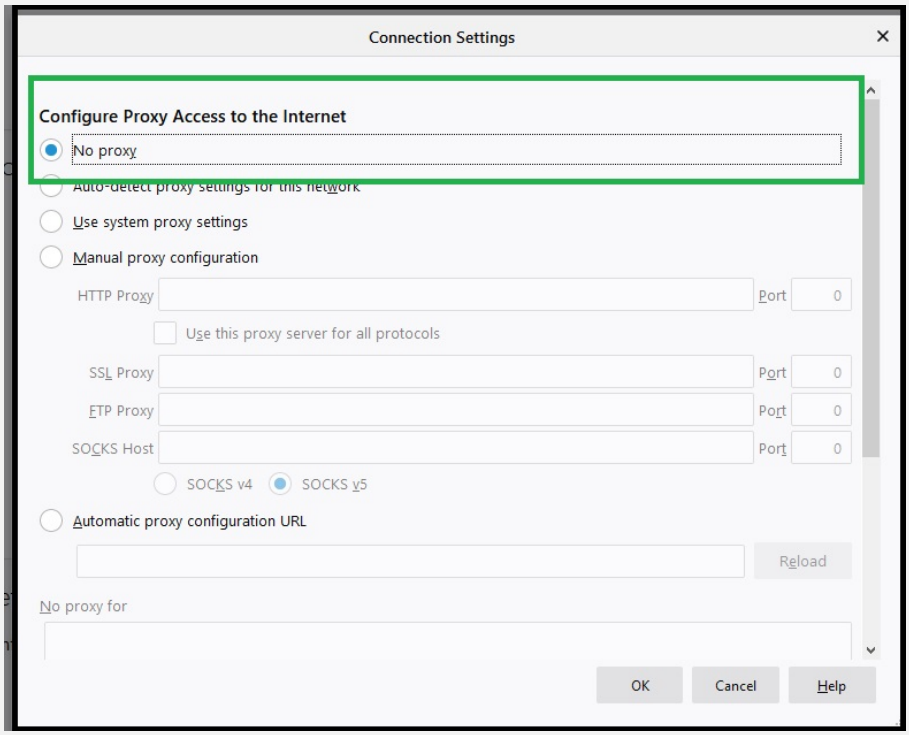


Abbildung 4: Deaktivieren der Proxy-Einstellung für den Browser `Mozilla Firefox`.

3 Cookie-based Authentication: Zustandsbehaftete Web-Authentifizierung mit Cookies

Zielsetzung

- Studieren der Cookie-basierten Authentifizierung für Websites und webbasierte Dienste
- Verstehen der Grundlagen: Erkennen der Notwendigkeit einer zustandsbehafteten Webauthentifizierung für das HTTP-Protokoll & Aufbau eines Cookies und Inhalt
- Schrittweise Analyse des Ablaufs der Cookie-basierten Authentifizierung

In diesem Kapitel werden wir Cookie-based Authentication mithilfe einer einfachen Beispiel-Webapplikation testen und nachvollziehen. Cookie-basierte Authentifizierung galt lange als State-of-the-Art und bewährte Methode für die Benutzerauthentifizierung.

Cookies speichern zustandsbehaftete Informationen für das zustandslose HTTP-Protokoll, oft Anmeldeinformationen. Damit wird ein gravierender Nachteil von HTTP umgangen. HTTP ist aus Performance-Gründen zustandslos. Der Server speichert die aktiven Sessions bei sich lokal ab. Wie die Sessions gespeichert werden, ist dabei nicht entscheidend. Beliebte Beispiele sind Textdateien oder Datenbanken. Clientseitig wird ein Cookie angelegt, welches einen Session Identifier (SID) enthält. Daher wird diese Art der Authentifizierung Cookie-based Authentication genannt.

3.1 Applikation öffnen

1. Öffnen Sie die Hauptseite der Cookie-based Authentication-Demo mit **Google Chrome**: http://localhost
2. Mit der F12-Taste können Sie die Developer Tools öffnen. Diese werden wir für die Analyse des Login-Verfahrens verwenden. Interessant für uns sind die Tabs Network (siehe folgende Abbildung) und Application.

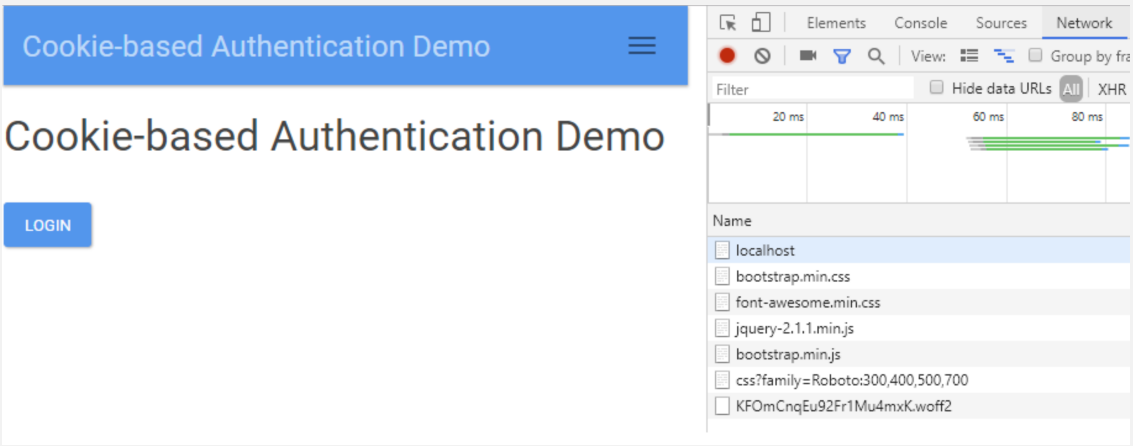


Abbildung 5: Screenshot der `Cookie-based Authentication App` mit `Google Chrome Developer Tools`

Im `Network-Tab` sehen wir alle HTTP-Aktionen, die vom Browser beim Aufruf einer Webpage durchgeführt werden. Eventuell ist ein Refresh der Website nötig, damit Sie die Daten oben sehen.

Im Beispiel oben wurde die Hauptseite des Web Servers `localhost` angefragt. Danach wurden noch weitere Ressourcen wie `CSS`-Bibliotheken und die JavaScript Library `Bootstrap` geladen. Diese "Zusatzressourcen" interessieren uns in den kommenden Analysen eher weniger. Bedeutet, Sie werden diese zwar jeweils im `Network-Tab` sehen, können sie aber ignorieren.

Für die Analyse werden wir nicht nur die einzelnen Interaktionen an sich, sondern auch die `HTML Header` betrachten müssen. Per Klick auf die jeweilige Seite können wichtige Details wie `Request`-Methode und `Response`- und `Request` Headers betrachtet werden.

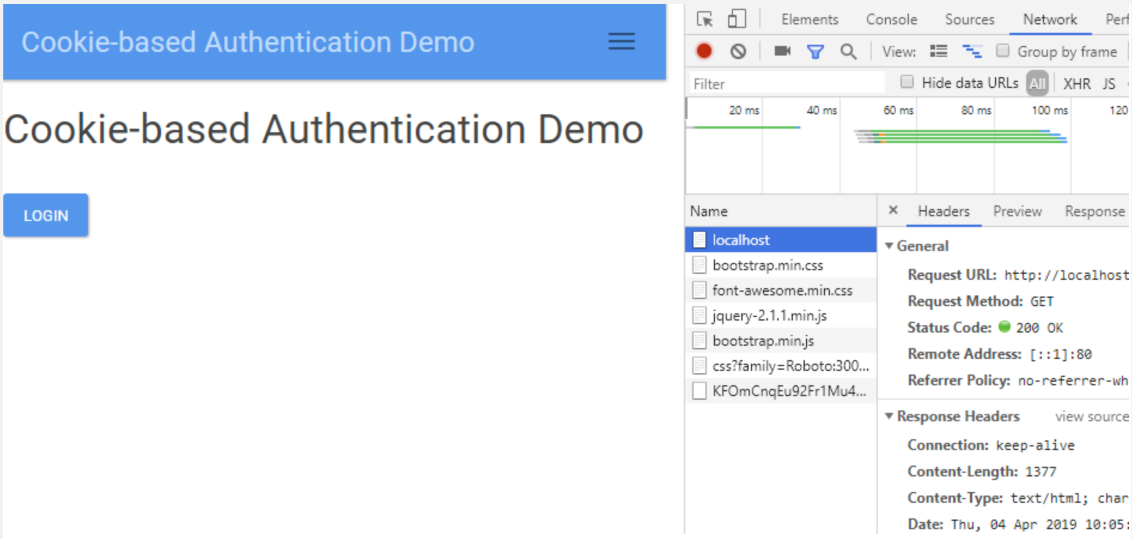


Abbildung 6: HTTP Headers (rechts unten) in den `Google Chrome Developer Tools`

3.2 Login durchführen

1. Klicken Sie auf `Login`. Sie werden nun auf die Login-Seite weitergeleitet. Was Sie hier sehen, ist ein gewöhnliches Login-Formular, wie Sie es auch aus dem Modul `WEBT` kennen.

2. Welche `HTTP-Methode` wird verwendet, um Daten vom Browser an einen Web-Server zu übertragen? Sprich, mit welchem Vorgang werden die Logindaten an den Server übertragen?
   Hinweis: Schauen Sie sich den Quelltext der Website an. Sie finden die Lösung im `action`-Parameter des HTML-Login-Formulars.

   Mit einer POST-Anfrage (POST-Request) an `/auth` werden die Daten des Formularelements an den Server gesendet.

3. Sehen Sie sich vor dem Einloggen noch das unten stehende Sequenzdiagramm an. Dies zeigt, was beim Klicken auf den Login-Button passieren wird.

Abbildung 7: Sequenzdiagramm zum Beginn des Ablaufs der `Cookie-based Authentication`

4. Wechseln Sie zuerst in den `Application-Tab`.
5. Überprüfen Sie, ob von der Website http://localhost Cookies bestehen. Dazu im `Storage` -Menü die Cookies aufklappen und verifizieren, dass keine Einträge für http://localhost bestehen.

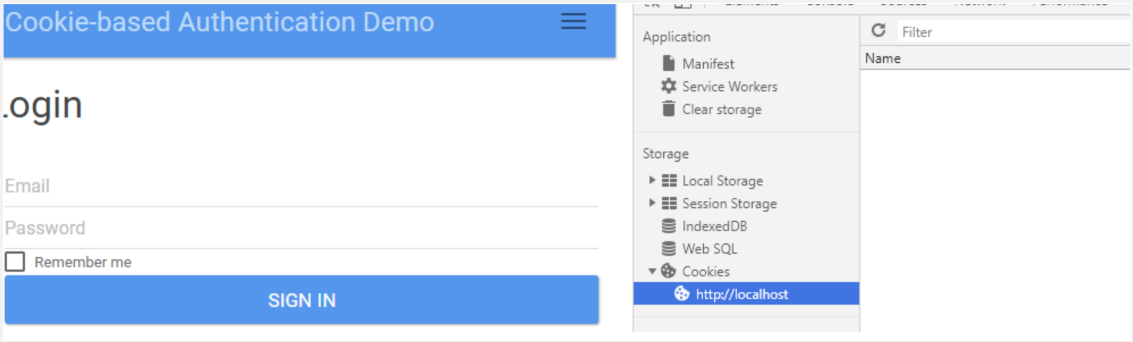


Abbildung 8: Auflisten der vorhandenen Cookies in den `Google Chrome Developer Tools`

6. Loggen Sie sich mit den folgenden Login-Daten ein. Die Login-Daten sind lokal auf dem Server gespeichert.

```
User:     tino
Passwort: 12345
```

3.3 Analyse

Nach erfolgreichem Login sehen Sie das `Dashboard` vor sich. Nun werden wir analysieren, ob wir die Vorgänge im Sequenzdiagramm auch hier nachvollziehen können.
Wir befinden uns immer noch im `Application-Tab` und sehen, dass wir jetzt ein Cookie für http://localhost besitzen. Anhand des Namens können wir erraten, dass es sich um ein Session-Cookie handeln könnte.

1. Wie lautet der Name des Cookies?

   `connect.sid`

2. Wechseln Sie wieder in den `Network-Tab` der `Developer Tools`.

   Sie sehen, dass zwei Ressourcen angefragt wurden: `/auth` und `/dashboard`. Konzentrieren wir uns zuerst auf `/auth`. Betrachten Sie `Request` und `Response Header` und beantworten Sie die folgenden Fragen.

3. Wo wird das Session-Cookie gesetzt?
   Dokumentieren Sie Ihre Session ID.

   Als Antwort auf die Request-Anfrage nach `/auth` mit den Login-Credentials mit Passwort und Benutzername, erhalten wir eine Antwort (`HTTP Response`). Im Response Header wird das Session-Cookie übermittelt, das der Server generiert hat. Dort ist unter `set-cookie` definiert unter welchem Namen, mit welchem Wert und für welchen Pfad (auch Verfallsdatum) der Browser ein Cookie setzen soll.

4. Warum wurden Sie *nach* dem `POST` der Login-Daten nach `/auth` auf eine andere Website (`/dashboard`) umgeleitet? Woher kam diese Instruktion?
   Hinweis: Schauen Sie sich die `Response Headers` des `HTTP POST` zu `/auth` an.

   Der Redirect wird vom HTTP-Status-Code 302 ausgelöst, welcher als Antwort auf den HTTP-Request vom Webserver zurückgesendet wird. HTTP-Antwortstatuscodes geben an, ob eine bestimmte HTTP-Anfrage erfolgreich abgeschlossen wurde. Sie werden stets in der ersten Zeile der HTTP-Response (= Server-Antwort) in der Form `HTTP/1.1 Statuscode` übermittelt, wobei `Statuscode` aus einer dreistelligen Zahl und einer kurzen Beschreibung besteht. HTTP-Code `302` besagt, die Seite wurde temporär verschoben. Der Browser soll statt der angefragten URL direkt nun eine weitere URL als Ersatz für seine Anfrage anfragen. Dazu steht im Response-Header zusätzlich die Anweisung `Location: /dashboard`. HTTP-Redirects enthalten in der Regel im Response-Header des Servers Statuscodes, die mit der Ziffer 3 beginnen, sowie das Location-Feld, das die URL angibt, zu der die Umleitung erfolgen soll.
   `HTTP/1.1 302 Found; Location: /dashboard; set-cookie: connect.sid=s%3A7aad188b-f1d6-4f33-a682eb3bb4100ebd.Ff8LeSzAusCqvnEBW51NtiDKtWS2CWo9FuZvcUJWe4o; Session ID: 7aad188b-f1d6-4f33-a682-eb3bb4100ebd`

5. Als Nächstes schauen wir uns die `/dashboard`-Anfrage an. Betrachten Sie wiederum `Request` und `Response Header`.

   Wie hat der Client bei der Dashboard-Anfrage beweisen können, dass er authentifiziert ist?
   Hinweis: Schauen Sie sich den `Request Header` an.

   Der Client hat das Cookie im `Cookie`-Feld im HTTP Request Header mitgesendet, welches eine ID, die der Server für ihn generiert hat, enthält.
   Der Server weiss, wem diese ID gehört und ob dieser eingeloggt ist. Cookie: `connect.sid=s%3A7aad188b-f1d6-4f33-a682-b3bb4100ebd.Ff8LeSzAusCqvnEBW51NtiDKtWS2CWo9FuZvcUJWe4o`

Wie Sie bereits in den Theoriefragen gelernt haben, ist `HTTP` ein `zustandsloses Protokoll`.

- Daraus ergibt sich der Vorteil, dass Requests unabhängig sind und auch parallel abgearbeitet werden können. HTTP-Requests können sehr schnell und effizient beantwortet werden.
- Es müssen aber alle Informationen beim Request mitgegeben werden, damit der Request vollständig bearbeitet werden kann. Dementsprechend muss bei jeder Anfrage dem Server erneut bewiesen werden, dass der Client bereits authentifiziert ist.

6. Das nachfolgende Sequenzdiagramm zeigt, wie bei jedem Request die Cookie-Session-ID mitgesendet wird. Mit Hilfe der `Session-ID` kann der Webserver Informationen wie Login-Daten bei sich nachschlagen und beim Bearbeiten des Requests verwenden.

Abbildung 9: Requests mit eingeloggter Session: Das Sequenzdiagramm zeigt wie bei jedem Request der Session-ID-Cookie mit `Cookie: session=...` mitgesendet wird.

7. Wir können diesen Umstand nochmals überprüfen, indem wir das Dashboard neu laden (`F5`) und wieder den `Request Header` anschauen. Sie stellen fest, die Session-ID wird mit jeder Anfrage mitgeschickt.

Wie Cookies gegen Angriffe geschützt werden können, ist Teil des Moduls `Secure Programming` und wird an dieser Stelle nicht weiter behandelt.

8. Schauen Sie sich nun die serverseitigen Cookie-Informationen im Dashboard an.

   Normalerweise sind diese Informationen für den Client nicht sichtbar. Hier wurden die serverseitigen Session-Daten für das Verständnis grafisch im Browser dargestellt. Beantworten Sie die folgenden Fragen.

9. Wie lange ist das Cookie gültig? Schreiben Sie sich die Zeitdauer auf. Beachten Sie die angegebene Zeitzone.

   Der Timestamp muss mit Zeitzone (`...Z`) beachtet werden. Das `Z` am Ende bedeutet, dass die Zeit in der Zeitzone UTC angegeben ist. Wenn man dies berücksichtigt und auf lokale Zeit umrechnet, ist das Cookie 12h gültig. `2021-10-09T01:15:52.503Z`

10. Was bedeutet die Zahl unter `__lastAccess`? In welchem Format ist diese Zahl?

    Zeitangabe des letzten Zugriffs: in meinem Fall die Anmeldezeit (08:58). Die Zeitangabe wird als Unix-Zeitstempel angegeben. Das Format `Unix-Zeitstempel` (`unix timestamp`) ist definiert als die Sekunden seit 1. Januar 1970 bis zum gewünschten Zeitpunkt.

11. Laden Sie die Seite nochmals neu.

    Sie werden feststellen, dass die Gültigkeit des Cookies, welche im Dashboard angezeigt wird, sich mit Ihrer Interaktion mit der Website erhöht hat. Mit diesem Mechanismus wird verhindert, dass aktive Verbindungen nach Ablauf des Cookies ausgeloggt werden.

12. Folgende Tabelle beschreibt zusammengefasst die Vor- und Nachteile von `Cookie-based Authentication`. Sie werden die Tabelle später wieder benötigen, wenn Sie die anderen Technologien mit dieser hier vergleichen.

    Tabelle 1: Vor- und Nachteile von `Cookie-based Authentication`

Vorteile	Nachteile
Eine Authentifizierung ist nur zu Beginn beim Besuch der Webseite erforderlich.	Eigener Registrierungsvorgang muss implementiert werden.
Einfach zu implementieren.	Login-Daten lokal gespeichert → müssen geschützt werden.
	Username / Passwort als Login eher unsicher
	Verschiedene Angriffsvektoren vorhanden wie Phishing, Session Hijacking, etc.

13. Beenden Sie nun die `Cookie-based Authentication`-Demo-Applikation. Schlies-

sen Sie dazu einfach das Kommandozeilen-Fenster `cmd.exe`, in welchem Sie die Applikation gestartet haben.

Zusammenfassung

Sie haben sich mit Cookie-basierter Web-Authentifizierung beschäftigt.

- Sie wissen, wie Cookies bei der Web-Authentifizierung eingesetzt werden und dass sie das Problem lösen, dass das HTTP-Protokoll zustandslos ist.
- Sie kennen den Ablauf von Cookie-basierter Web-Authentifizierung.
- Sie können die Developer Tools eines Webbrowsers nutzen und das HTTP-Protokoll analysieren.

Sie haben HTTP-Cookies als im Browser gespeicherte Text-Informationen kennengelernt und verstanden, dass ein Nutzer identifiziert (Session ID) oder ein Login für eine Webanwendung abgespeichert werden kann.

- Anhand der Informationen im Browser können Sie die Inhalte eines Cookies deuten.
- Sie wissen wie ein Cookie aufgebaut ist und aussieht.
- Sie kennen die `Vor-` und `Nachteile` von `Cookie-basierter Authentifizierung`.

4 Open Authorization (OAuth 2.0) mit OpenID Connect (OIDC): Sichere API-Autorisierung für Desktop-, Web- und Mobile-Anwendungen

In diesem Versuch werden Sie sich mit dem Autorisierungsprotokoll `OAuth 2.0` im Zusammenspiel mit `OpenID Connect (OIDC)` befassen. `OIDC` ist eine Authentifizierungsschicht, die auf OAuth 2.0 basiert; also eine Schicht oberhalb des `OAuth`-Protokolls. Im Folgenden sprechen wir primär von `OAuth 2.0`, wo wir `OIDC` in diesem Versuch als integriert betrachten.

Für diesen Versuch wurden zwei Demo-Anwendungen – API-Server `server.js` und Authentication App `app.js` – auf der Windows Workstation abgelegt. Mithilfe von diesen werden Sie das Protokoll `OAuth 2.0` für die Authentifizierung (OIDC) und Autorisierung analysieren.

4.1 Starten der Applikationen

1. Starten Sie die beiden Applikationen von zwei separaten Kommandozeilen aus:

```
C:\Windows\system32> cd \Users\cisco\Documents\web-
    authentication\oauth-api
C:\Users\cisco\Documents\web-authentication\oauth-api> npm
    start
```

```
C:\Windows\system32> cd \Users\cisco\Documents\web-
    authentication\oauth-authentication
C:\Users\cisco\Documents\web-authentication\oauth-
    authentication> npm start
```

Das Backend `server.js` im Ordner `OAuth-API` müssen Sie als Administrator ausführen. Dies deswegen, da wir einen Eintrag im lokalen Hosts-File machen (siehe URL im späteren Verlauf). Öffnen Sie entsprechend `cmd.exe` / `PowerShell` als Administrator (`Run as administrator`).

Es ist zwingend notwendig, die Anwendungen in dem Ordner zu starten, in dem alle notwendigen Dateien und Daten liegen. Daher ist der Wechsel in den Ordner `web-authentication\OAuth-API` bzw. `web-authentication\oauth-authentication` zwingend notwendig.

4.2 Konfiguration des Zed Attack Proxys (ZAP) mit Mozilla Firefox

1. Öffnen Sie `Firefox` und öffnen Sie die `Authentication App`. Diese läuft auf Ihrem PC unter `securityathslu.ch`

   ```
   securityathslu.ch
   ```

2. Klicken Sie auf den Login-Button.
3. Registrieren Sie auf der Maske einen neuen Benutzer (beliebige Credentials).
4. Nach der Registrierung werden Sie automatisch angemeldet, loggen Sie sich zu diesem Zeitpunkt gleich wieder aus.
5. Öffnen Sie das Programm Zed Attack Proxy (ZAP) auf Ihrer Workstation. *Speichern Sie die ZAP-Session nicht*.

Damit Sie den Netzwerkverkehr mit der Third Party-Website `Auth0`[1] als Identity-Provider mit Autorisierungsserver mitlesen können, müssen Sie einen SSL/TLS-Proxy verwenden. Dies geschieht logischerweise deswegen, da die Authentifizierung über den Identity Provider per HTTPS verschlüsselt wird und normalerweise nicht einsehbar ist. Entsprechend müssen wir die Verbindung aufbrechen, um die Daten im Klartext zu sehen und zu studieren. Wir führen genau genommen eine `Man-in-the-Middle (MitM)-Attacke` durch, um den Verkehr einsehen zu können. Sie werden sich in einem späteren Labor noch intensiver mit den Themen `SSL/TLS-Proxy` und `Man-in-the-Middle (MitM)` auseinandersetzen.

[1] `Auth0` - Authentication-as-a-Service, http://auth0.com/

Für den Versuch müssen wir das Zertifikat des ZAP-Proxys in Firefox importieren, damit eine sichere Kommunikation aufgebaut werden kann. Wir führen eine Man-in-the-Middle (MitM)-Attacke durch. Dies werden Sie in den folgenden Schritten vornehmen.

6. Klicken Sie im ZAP auf `Tools → Options... -> Network → Server Certificates`. Sie sollten das Root CA-Zertifikat sehen.
7. Speichern Sie dieses auf Ihren Desktop ab (`Save`).

8. Wechseln Sie wieder zum Firefox und öffnen Sie dort die `Settings`. Wählen Sie den Reiter `Privacy & Security`. Navigieren Sie zu `Security` → `Certificates` → `View Certificates ...`.
9. Importieren Sie das exportierte Root CA-Zertifikat vom ZAP in die `Zertifizierungsstellen (Authorities)` von Firefox.
10. Wählen Sie beim Import des Zertifikats die Option `Trust this CA to identify websites` (bzw. `Dieser CA vertrauen, um Website zu identifizieren`).

11. Was bezwecken Sie mit diesem Vorgang (`Import Root CA`)? Wieso ist dieser Schritt notwendig?

Der Firefox Browser wird dadurch die angebotene (gefakte) Verschlüsselung vom OWASP ZAP akzeptieren und dieser vertrauen. Firefox würde sonst die Verbindung ablehnen, resp. als MitM-Angriff deklarieren. Mit dem importierten Root-Zertifikat (Root-CA) vom ZAP akzeptiert der Firefox beliebige Zertifikate, die von diesem Root-CA abstammen, weil er diesem vertraut. Der ZAP kann somit als zwischengeschalteter Proxy verwendet werden, der die Verschlüsselung der angefragten, echten Webseite aufbricht und eine eigene Verschlüsselung zum Firefox hin vorgibt - er gibt sich in Richtung `Firefox` als die angefragte Webseite aus, als würde die verschlüsselte Verbindung direkt von der echten Webseite kommen. Der Vorteil ist, dass man die Kommunikation beim ZAP-Proxy entschlüsseln und einsehen kann. Webseiten-Anfragen und -Antworten werden beim ZAP entschlüsselt und aber erneut verschlüsselt; in Richtung Firefox jedoch mit seinem eigenen Zertifikat, damit er für beliebige URLs on-the-fly die richtige URL in das Zertifikat schreiben kann und sich so als beliebige Webseiten ausgeben kann. Siehe auch https://it-forensik.fiw.hs-wismar.de/index.php/SSL_Inspection.

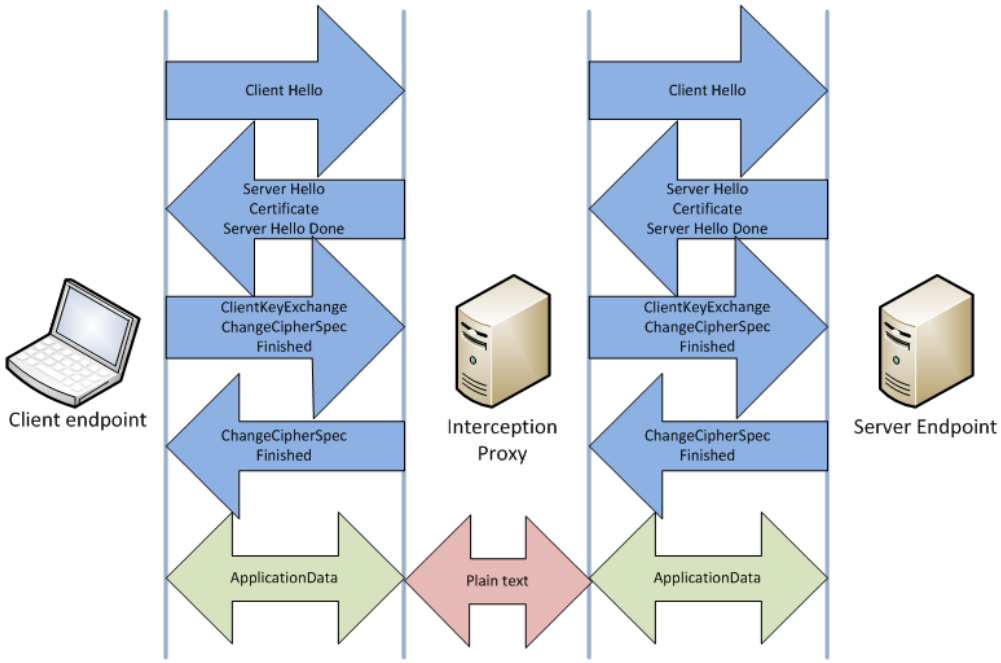


Abbildung 10: Beispiel-Aufbau mit Proxy.

12. Wechseln Sie nun in den Firefox: `Settings` → `General` → `Network Settings`. Setzen Sie Ihren Proxy gemäss Abbildung 11.

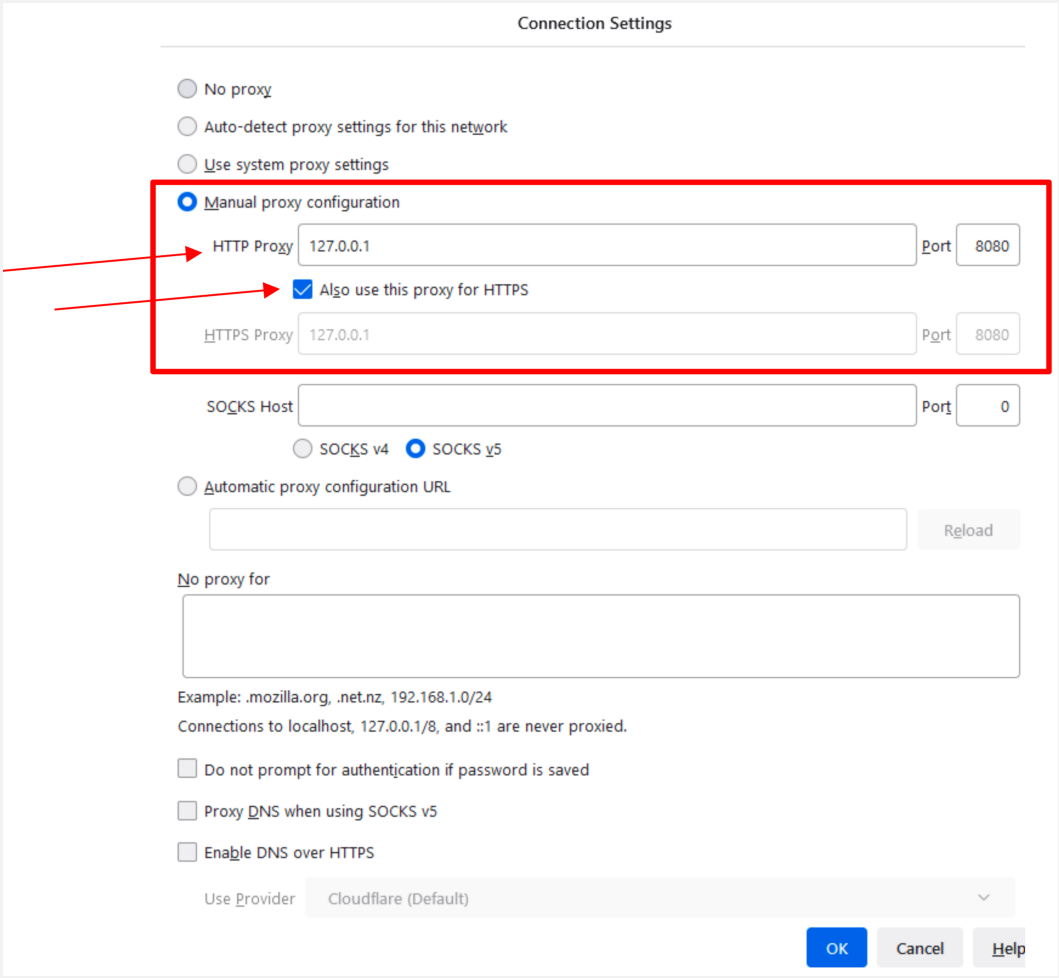


Abbildung 11: Proxy-Einstellungen für die Nutzung des lokalen SSL/TLS-Proxys in Mozilla Firefox für HTTP und HTTPS.

Sie haben nun alle Schritte unternommen, um per SSL/TLS-Proxy die Kommunikation des Browsers mitlesen und analysieren zu können.

4.3 Analyse des Ablaufs von OAuth 2.0

Sie werden im Folgenden den Workflow einer Authentifizierung und Autorisierung mit `OAuth 2.0` über `Auth0.com` (Identity Provider, https://auth0.com) anhand einer normalen serverseitigen Webanwendung betrachten. Der Workflow ist in Abbildung 12 abgebildet.

Beachten Sie, dass hier der *generelle Workflow* und nicht alle einzelnen HTTP-Requests und -Responses abgebildet sind.

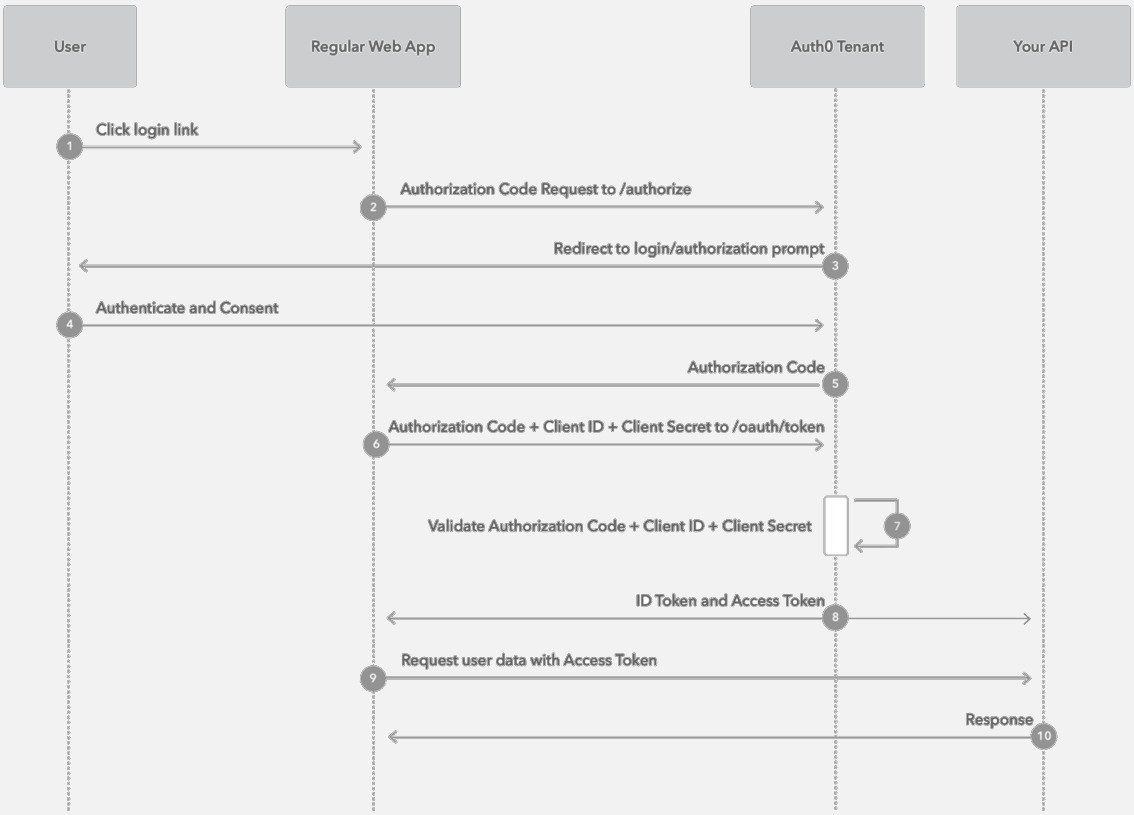


Abbildung 12: `OAuth 2.0`-Ablauf im Sequenzdiagramm (Quelle: https://auth0.com/docs/flows/concepts/auth-code#how-it-works)

In den folgenden Abschnitten erläutern wir jeden einzelnen Schritt und besprechen, wie er funktioniert.

4.3.1 Schritt ①: `Click login link`

Beschreibung: Der Benutzer klickt in der regulären Webanwendung auf `Login`.

1. Wechseln Sie zu Firefox auf die Authentication App.
2. Starten Sie im ZAP eine neue Aufzeichnung (Icon `weisses Blatt` → `Neue Session`).

> Sie müssen die ZAP-Session nicht speichern → `I do not want to persist this session ...`.

3. Deaktivieren Sie im ZAP das Heads Up Display (HUD). Sie sehen das HUD-Radarsymbol in der Symbolleiste des ZAP-Proxys. Wenn das Symbol nicht ausgewählt ist, wird das HUD nicht angezeigt, wenn Sie Ihren Browser verwenden.

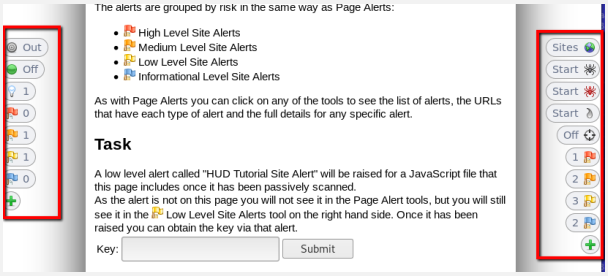


Abbildung 13: Webbrowser mit angezeigtem Heads Up Display (HUD) an der rechten und linken Seite. Wir deaktivieren diese Anzeige für den Rest des Labors.

4. Machen Sie in Firefox einen Refresh der Login-Seite. Stellen Sie sicher, dass die URL `securityathslu.ch` und *nicht* `localhost` ist.

> Falls Sie automatisch zu HTTPS weitergeleitet werden, löschen Sie den Browsercache in `Settings` → `Privacy` & `Security` → `Clear Data ...` → `Clear`.

5. Schauen Sie sich das Ergebnis im ZAP an.

6. Welche weiteren Webseiten und Web-Ressourcen wurden neben http://securityathslu.ch noch geladen? Um welche Sachen handelt es sich dabei? Welche der Seiten oder Ressourcen werden möglicherweise direkt für `OAuth` verwendet? Googeln Sie gegebenenfalls nach den Namen im Internet, insbesondere `socket.io.js`

   Wichtig ist `/socket.io/socket.io.js`. Diese Bibliothek wird oft im Zuge von OAuth2.0 verwendet. Sie vereinfacht die Arbeit mit Websockets unter Verwendung von Node.js. *Hinweis:* Dies wird später oft im Zap zu sehen sein, da die Web-Anwendung hierüber die Kommuikation asynchron im Hintergrund durchführen kann, ohne dass der Nutzer irgendwo klicken muss. Unter anderem werden ID und Access Token mit Hilfe dieser Bibliothek fortlaufend aktualisiert. Im HTML der ersten Abfrage werden zusätzliches CSS und JS-Bibliotheken geladen.

   1. google-fonts - verwendete Schriftarten
   2. bootstrap - css/js framework
   3. jquery - freie JavaScript-Bibliothek, die Funktionen zur DOM-Navigation und -Manipulation zur Verfügung stellt.
   4. favicon - Icon für das Browser-Tab

Für diesen Versuch interessiert uns nur der Traffic für http://securityathslu.ch und der Third Party-Seite (Auth0.com (https://auth0.com)). Alle anderen Ressourcen können Sie jeweils per Rechtsklick → `Exclude from` → `Proxy` ausschliessen (jQuery, Bootstrap, Google Fonts).

7. Schauen Sie sich den GET-Request wie auch die Response auf http://securityathslu.ch/ an (im ZAP-Tab `History`). Im weiteren Verlauf werden Sie sich so durch den Workflow durchspielen.

8. Klicken Sie nun den Login-Button.

4.3.2 Schritt ②: `Authorization Code Request to /authorize`

Beschreibung: Das SDK von `Auth0.com` leitet den Benutzer zum Autorisierungsserver (`/authorize endpoint`) um.

1. Wählen Sie im ZAP den Request auf `http://securityathslu.ch/login`. *Hinweis:* Der Request gehört genau genommen noch zu Schritt ①.

2. Wohin zeigt die Response? Was wird mit der `audience` definiert? Was wird sonst noch mitgesendet?

   `hslu-oauth.eu.auth0.com/authorize[...]audience=https://hslu-oauth-demo-api.ch`

   1. Dies ist die Umleitung der Web-Anwendung auf den Auth0-Server für das Durchführen des Logins. Hiermit wird dem Browser angezeigt, welche URL als nächstes für den Login aufgerufen werden soll.
   2. `audience=...` (Deutsch: Zielgruppe) gibt an, für welche Ressource/API ein Token nach dem Login ausgestellt werden soll. Streng genommen gibt es in OAuth 2.0 und OpenID Connect zwei Arten von `audience`-Konzepten:
      - *OpenID Connect-ID-Token-Audience:* OpenID Connect-ID-Token sind für extere Ressourcen und Teilnehmer bestimmt. Diese `audience`-Definition eines OpenID Connect ID-Tokens definiert, welche Clients es akzeptieren sollen.
      - *OAuth 2.0-Access Token (Zugriffstoken)-Audience:* Die Definition eines OAuth 2.0-Zugriffstokens definiert die Endpunkte oder Server, an denen das Token verwendet werden kann. Mit Hilfe des Access Tokens kann die Web-Anwendung später (oder theoretisch auch jeder Besitzer des Tokens) eine Anfrage an eine externe Ressource/API/URL stellen, ohne sich dort authentifizieren zu müssen.

3. Schauen Sie sich jetzt den entsprechenden Request von Schritt ② auf `https://hslu-oauth.eu.auth0.com/authorize?...` an.

Wie Sie sehen sollten, fragen Sie hiermit die Login-Maske des Identity Providers (= Auth0.com (http://auth0.com)) an (siehe Workflow in Abbildung 12).

4. Betrachten Sie die Parameter des Redirects. Hier eine kurze Erklärung dazu:

   - *State*
     Berechtigungsprotokolle stellen einen Zustandsparameter zur Verfügung, mit dem Sie den vorherigen Zustand Ihrer Anwendung wiederherstellen können. Der Hauptgrund für die Verwendung des Zustandsparameters ist die Reduzierung von CSRF-Angriffen.[1]

   - *Client*
     Die Client-ID ist auf `Auth0` (http://auth0.com) in den Einstellungen vorkonfiguriert worden und in der Webanwendung entsprechend im JavaScript-Code einprogrammiert, siehe Abbildung 14.

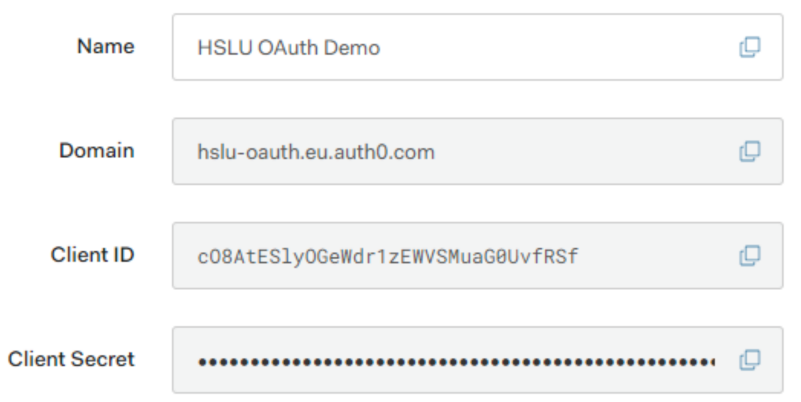


Abbildung 14: `Auth0`-Applikations/API-Verwaltung. (Quelle: https://auth0.com)

   - *Audience:* API-Endpoint

   - *Scope*
     Im Scope wird die Berechtigung für den Zugriff von User-Informationen definiert.

[1] https://auth0.com/docs/protocols/oauth2/oauth-state#csrf-attacks

4.3.3 Schritt ③: `Redirect to login/authorization prompt`

Beschreibung: Der Auth0-Autorisierungsserver fragt bei der Web-Anwendung an, ob der Benutzer authentifiziert werden soll.

1. Schauen Sie sich die Response zum entsprechenden Request aus Schritt ② an. Hier finden Sie die Login-Eingabemaske, die dann im Browser angezeigt wird. Dies entspricht Schritt ③.

4.3.4 Schritt ④: `Authenticate and Consent`

Beschreibung: Der Nutzer gibt seine Zugangsdaten ein und authentifiziert sich über die Login-Eingabemaske im Browser. Das Passwort und die E-Mail-Adresse werden innerhalb der Login-Form an den Authorisierungsserver übertragen.

1. Melden Sie sich mit dem zuvor registrierten Benutzer an.
2. Schauen Sie sich den Request auf `https://hslu-oauth.eu.auth0.com/usernamepassword/login` an.
3. Was sehen Sie im Request aus Schritt ④? Was sehen Sie in der Response?

   Request:

   - `Referer: https://hslu-oauth.eu.auth0.com/login;`
   - `state`, `client`, `protocol`, `audience`, `response_type`, `redirect_uri`, `scope`.
   - `Auth0-Client: eyJuYW1l...`, `Cookie: _csrf=Y7fXXq0j...`

   Im `Request` werden Client-ID (`client`), Umleitungs-URL nach dem Login (`redirect_uri`), Response Type (hier `Authorization Code`), Scope (`scope`), State (`state`), Zielgruppe (`audience`), Benutzername und Passwort mitgesendet.
   In der `Response` kommt ein Formular mit drei Input-Feldern zurück. In diesen Input-Feldern sind erneut alle Infos vom Request drin, inklusive dem Auth0-Client-Feld. In der Response wird die redirect-URL zur nächsten Seite zurückgegeben.

4.3.5 Schritt ⑤: `Authorization Code`

Beschreibung: Der Auth0-Autorisierungsserver sendet den Benutzernamen mit einem Autorisierungscode, der für eine einmalige Verwendung geeignet ist, zurück zur Web-Anwendung, um Bestätigung zu bitten.

4.3.6 Schritt ⑥: `Authorization Code + Client ID + Client Secret to /token`

Beschreibung: Die Web-Anwendung sendet den erhaltenen Authorization Code zusammen mit der vorweg konfigurierten Client ID und dem Client Secret der Anwendung an den Autorisierungsserver, um den Login-Vorgang zu bestätigen.

4.3.7 Schritt ⑦: `Validate Authorization Code + Client ID + Client Secret`

Beschreibung: Der Autorisierungsserver überprüft den gelieferten Code, die gelieferte Client ID, wie auch das Client Secret. Falls alles korrekt ist, erstellt der Autorisierungsserver das ID und Access Token.

1. Folgen Sie dem Stream an Requests und Responses im Tab `History` bis zum Request `http://securityathslu.ch/user` im ZAP-Proxy.

2. Erklären Sie in Ihren eigenen Worten den Ablauf von der Anmeldung auf der Login-Maske bis hin zum Redirect auf `http://securityathslu.ch/user`.
   Hinweis: Orientieren Sie sich an der Abbildung 12 zum Workflow.

   1. Schritt ④: Login: Anfrage mit POST-Request auf `/login`: Senden der Credentials an `hslu-oauth.eu.auth0.com`; dann POST: redirect, `login/callback`.
   2. Schritt ⑤: Nach dem Login wird der Authorization-Code von `Auth0.com`, dem Identity-Provider, direkt im Hintergrund an die Web-Anwendung gesendet. Dies geschieht, um bei der Web-Anwendung nachzufragen, ob der Nutzer für die Web-Anwendung authentifiziert werden soll.
   3. Schritt ⑥: Da vorweg ein Client-Secret und eine Client-ID auf `Auth0.com` konfiguriert wurden, ist nun der nächste Schritt, dass die Web-Anwendung den Anmeldeversuch mit dem zuvor festgelegten Client-Secret und -ID bestätigt. Dazu sendet die Web-Anwendung den Authorization-Code zusammen mit der Client-ID und dem Client Secret an `Auth0.com`. Dies geschieht im Hintergrund und ist nicht im ZAP-Proxy einsehbar.
      (Anmerkung: Hier wäre zur Sicherheit Authorization Code Flow mit Verwendung von Proof Key for Code Exchange zu brauchen).
   4. Schritt ⑦ und ⑧: Der Server validiert das `Client Secret` und sendet nur bei Übereinstimmung das ID und Access Token an den Client (ID Token nur aufgrund der Konfiguration im Scope (= “openid”)).
   5. Im ZAP-Proxy sieht man nun, dass die Web-Anwendung im Browser beim Nutzer nach Erhalt der Tokens folgendes bis zum abschliessenden `securityathslu.ch/user` macht:

      - GET-Request: auf eigenen `/callback` mit 2 Parameter;
      - GET-Request: `/securityathslu.ch/user`; redirect: man wird abschliessend auf `securityathslu.ch/user` weitergeleitet; zum Erzeugen und Darstellen der erhaltenen Informationen (inkl. der Informationen über die erhaltenen Tokens) im Browser auf der Seite der Web-Anwendung.

   Im Anschluss hat die Web-Anwendung das ID Token und kann mit dem Access Token die API des externen Dienstes abfragen (Schritt ⑨ und ⑩) und das UI mit den Profil-Infos aus dem ID Token personalisieren.

4.3.8 Schritt ⑧: `ID Token and Access Token`

Beschreibung: Der Autorisierungsserver antwortet mit einem ID Token und einem Access Token an die Web-Anwendung.

1. *Wichtig:* Wie Ihnen auffallen wird, sehen Sie die Schritte zwischen ⑤ bis ⑧ nicht im ZAP-Proxy.

   - Welche Nachrichten können Sie generell im ZAP-Proxy sehen und von wem? Geben Sie die Kommunikationspartner aus unserem Szenario an, deren Verkehr man im ZAP-Proxy anschauen kann.
   - Welche Kommunikation ist NICHT im ZAP-Proxy zu sehen, aber für OAUTH notwendig?
   - Weshalb sehen Sie die Schritte ⑤ bis ⑧ nicht im ZAP-Proxy?
   - Wo könnte man somit das ID Token und das Access Token finden?

   Hinweis: Es handelt sich um eine serverseitige Anwendung.

   Der ZAP-Proxy entschlüsselt nur den Verkehr zwischen Browser und externen Webseiten. Folglich ist nur der Verkehr

   1) vom Browser zur Web-Anwendung und
   2) vom Browser zum `Auth0.com`-Authorisierungsserver zu sehen.

   Insbesondere ist *nicht* der Verkehr, der zwischen Web-Anwendung und `Auth0.com`, zwischen Web-Anwendung und Server und zwischen Server und `Auth0.com` getätigt wird, einsehbar. Die Web-Anwendung und der Server reagieren selbst automatisch auf die Antworten des Auth0-Tenants. Hier gibt es eine direkte Kommunikation zwischen Web-Anwendung/Server und `Auth0.com`, der nicht vom ZAP-Proxy, der im Browser eingestellt ist, gefiltert wird.
   Hinweis: Das Token ist dann aber auf dem Server in der Ausgabe auf der Kommandozeile ersichtlich. Diese Ausgabe wurde von der HSLU einprogrammiert, damit der Student die Informationen einsehen kann.

Damit der Workflow für die Schritte ⑤ bis ⑧ trotzdem nachvollzogen werden kann, wurde der Request auf `https://hslu-oauth.eu.auth0.com` und die Response von `https://hslu-oauth.eu.auth0.com/oauth/token` ins GUI im Webbrowser integriert. Die Webanwendung zeigt Ihnen die wichtigsten Inhalte der Kommunikation der Schritte ⑤ bis ⑧ im Browser an.

2. Schauen Sie sich im GUI der Webanwendung im Browser den `Validation Request` an. Sie sollten hier die drei Parameter `Authorization Code`, `Client ID` und `Client Secret` finden.

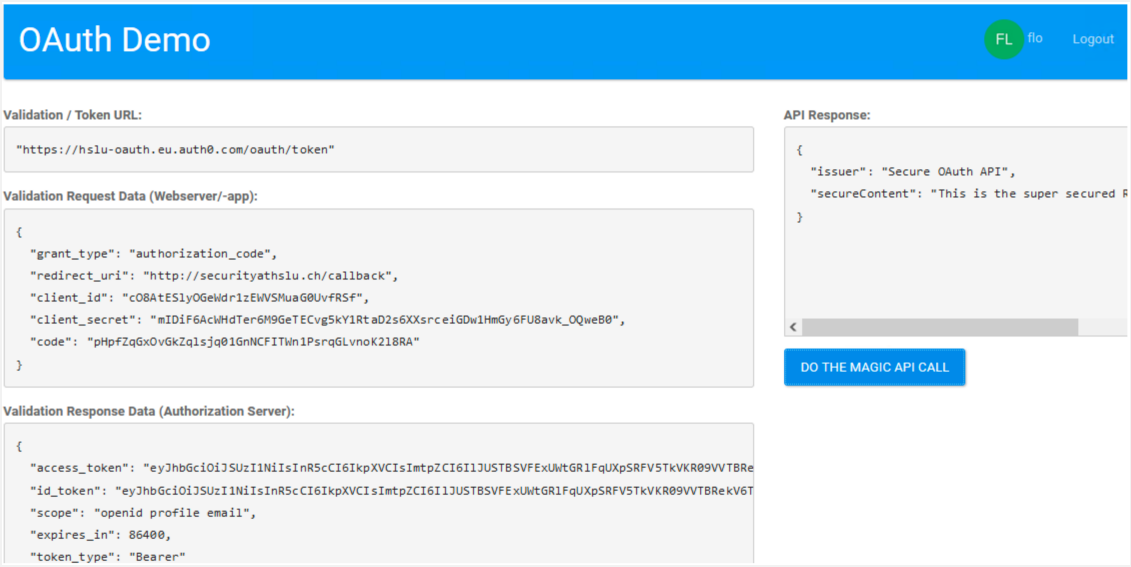


Abbildung 15: Screenshot der App mit den serverseitigen OAuth-Kommunikationsparametern im GUI im Webbrowser.

3. Werfen Sie anschliessend einen Blick auf die `Validation Response`. Hier finden Sie das `ID Token` wie auch das `Access Token`. Es handelt sich hierbei um `JSON Web Tokens (JWT)`.

Ein JSON Web Token (JWT) ist ein auf JSON basiertes und nach RFC 7519 genormtes Access Token. Das JWT ermöglicht den Austausch von verifizierbaren Claims. Es wird typischerweise verwendet, um in einem System mit einem Drittanbieter, die Identität eines Benutzers zwischen einem Identity-Provider und einem Service-Provider auszutauschen. Des Weiteren eignet sich JWT zur Implementierung einer Stateless Session, da alle für die Authentifikation benötigten Informationen in dem Token übertragen werden, muss die Sitzung nicht auf dem Server gespeichert werden.[1]

[1] https://de.wikipedia.org/wiki/JSON_Web_Token

4. Öffnen Sie in Firefox die Webseite www.jwt.io.
5. Kopieren Sie zuerst das `ID Token` und fügen Sie es in das Feld `Encoded` auf `jwt.io` ein. Achten Sie darauf, dass sie jeweils das komplette Token kopieren.
6. Was sehen Sie nach der Dekodierung?

   ID-Token: Wir sehen Profilinformationen über den User.

   1. Zu Beginn wird der Algorithmus für die Erstellung, Typ und Key ID definiert.
   2. Danach folgen die Informationen: Nickname, Name, Profilbild URL. Es beinhaltet auch die Zeit, zu welcher es ausgestellt wurde, ein Verfallsdatum, die Zugriffsgruppe (`audience`), für welche das Token gedacht ist und eine Session-ID.
   3. Die gesamte Information ist mit dem Key des Issuers mit RSA signiert. Unter Issuer ist angegeben, wer das ID Token signiert hat.

7. Machen Sie das Gleiche für das `Access Token`.

8. Was sehen Sie nach der Dekodierung?

Das Access Token enthält keine Benutzerinformationen.

1. Zu Beginn wird der Algorithmus für die Erstellung, Typ und Key ID definiert.
2. Danach folgen die Informationen. Unter anderem: Issuer (Ersteller), Subject, Audience, Ausstellungsdatum, Verfallsdatum, Authorized Party, sowie die Scopes. In den Audiences ist neben dem HSLU-Server (der auf dem PC von uns per Hand gestartet wurde) auch ein `/userinfo`-Endpoint erlaubt. Bei diesem können neben dem ID Token weitere Benutzerinformationen aktiv angefragt werden.
3. Ebenfalls kann auch hier die Signatur wieder verifiziert werden (Block am Ende).

4.3.9 Schritt ⑨: `Request user data with Access Token`

Beschreibung: Das Access Token spezifiziert die Rechte, die eine Web-Anwendung erhält, um beim Third-Party-Provider Informationen abzufragen oder einzugeben. Dies kann zum Beispiel das Recht sein, Informationen über den Nutzer (zB. sein Profilbild) beim Third-Party-Identity-Provider abzufragen.
Kurz gesagt: Die Anwendung kann mit dem Access Token eine API aufrufen und nutzen, die im Access Token angegeben ist. Im Folgenden machen wir dies exemplarisch mit dem Click auf `DO THE MAGIC API CALL`, siehe Schritt ⑩: `Response`.

4.3.10 Schritt ⑩: Response

Beschreibung: Wenn die API des Identity-Providers angefragt wird, antwortet diese mit den angeforderten Daten, wenn das Access Token beachtet wurde.

1. Klicken Sie nun den Button `DO THE MAGIC API CALL`.

Falls Sie folgende Fehlermeldung `error: error` erhalten, stellen Sie sicher, dass Sie auf der Webseite mit `HTTP` und nicht mit `HTTPS` verbunden sind. Bitte beachten Sie, dass Sie dafür das ZAP-HUD ausgeschaltet haben müssen, weil Sie sonst automatisch auf `https://...` umgeleitet werden, siehe Abschnitt 4.3.1.

Abbildung 16: Fehlermeldung bei falschem Zugriff auf die Web-Anwendung. Stellen Sie sicher, dass Sie sich per HTTP verbinden. Deaktivieren Sie das ZAP-HUD.

2. Schauen Sie sich den GET Request wie auch die Response an.
3. Was wird im HTTP-Header des Autorisierungs-Requests mitgesendet? Wie lautet der `Secure Content` von der API?

   HTTP-Request:

   - Header: `alg: RS256, typ: JWT, kid: RTI0RTQ1QkFFQjQzRDUyNEJGOUU0QzEzM0ZENzIzN0Y2NzU1ODU3MQ`
   - Payload: `iss: https://hslu-oauth.eu.auth0.com/, sub: auth0|6076e22cf9a81a006950143b, aud: [https://hslu-oauth-demo-api.ch, https://hslu-oauth.eu.auth0.com/userinfo], iat: 1618866816, exp: 1618953216, azp: cO8AtESlyOGeWdr1zEWVSMuaG0UvfRSf, scope: openid profile email`

   HTTP-Response (Antwort der API/`Secure Content`):

   - `issuer: Secure OAuth API`
   - `secureContent: This is the super secured Resource!`
   - → Als Antwort bekommen wir den Secure Content “This is the super secured Resource!”.

4. Schauen Sie sich das Access Token des Requests auf der Kommandozeilenausgabe des

API-Servers in der PowerShell-Konsole (oder CMD) an.

4.4 Abschluss `OAuth 2.0`

1. Stoppen Sie die beiden Anwendungen. Schliessen Sie die Kommandozeile.
2. Analog zur Vorteile/Nachteile-Tabelle im ersten Teil `Cookie-based Authentication` sollen Sie dies nun für OAuth 2.0 (inkl. OIDC für die Authentifizierung) festhalten. Nehmen Sie auch Bezug zu den folgenden Themen: APIs, Enterprise Applications, Federated Identity, Security, `Auth0.com` als Identity-as-a-Service (IDaaS).

Vorteile:

Deutlich sicherer als Cookie-Authentication, unter anderem Dank der Spezifiierung von folgendem:

1. Prüfung der Benutzer-Credentials durch einen externen Dienstanbieter (Identity-Provider) (nicht jede x-beliebige Webseite verwaltet die Passwörter) (Authentifizierung muss nicht unbedingt eigenständig implementiert werden und kann extern bezogen werden.),
2. der Möglichkeit von Addon-Authentifizierungs-Massnahmen des externen Identity-Providers: Zweifaktor-Authentifizierung (2FA), Einmalpasswörter, Hardware-Tokens.
3. Trennung zwischen Erlangen eines Tokens und dessen Einsatz, sowie begrenzter Gültigkeit. Es werden nur Tokens ausgetauscht, welche nach relativ kurzer Zeit ablaufen.
4. Dank (Quasi-)Standardisierung, gute Sicherheit und einfache Einbindung. Dies ermöglicht die Integration von OAuth in nahezu jede Lösung, einschliesslich mobiler und Desktop-Anwendungen, Websites und Plattformen und sogar in Browser-Plug-ins.
5. Federated Identities bieten, vor allem für grössere Unternehmen und Organisationen, welche eine Vielzahl an Applikationen betreiben, viele und unmittelbare Vorteile: “Ein Login für alle Dienste”: Single-Sign-On ist möglich, wenn der gleiche IdentityServer von allen Ressource-Servern verwendet wird. Identity-as-a-Service (IDaaS)-Provider wie `Auth0.com` profilieren sich zunehmend als valide Möglichkeit, das Identity und Access Management an Drittanbieter abzugeben - sowohl für Privatpersonen, wie auch Firmen oder Organisationen.
6. Darüber hinaus gibt es eine Reihe von vorgefertigten Lösungen, die an individuelle Bedürfnisse angepasst werden können; zudem kann OAuth mehrere Anfragen bearbeiten und eine Reihe von Benutzern gleichzeitig bedienen.
7. Weiter ist die Verwendung von Secure Sockets Layer (SSL), das die Datensicherheit zwischen Webserver und Browsern gewährleistet, vorgeschrieben. SSL sorgt dafür, dass die übertragenen Credentials zwischen Webserver und Browsern geheim bleiben.
8. OAuth ist ein einfacher Ansatz für eine starke Authentifizierung. So können Token bei Bedarf widerrufen werden (z. B., wenn verdächtige Aktivitäten festgestellt werden).

Nachteile:

1. Keine Vorgaben für die eigentliche Authentifizierung: Es ist immer noch möglich (einfache) Passwörter für die Authentifizierung mit OAuth zu nutzen. 2FA oder andere Alternativen sind “nur” optional möglich.
2. Abhängigkeit vom jeweiligen (externen) Identity-Dienstanbieter, evtl. ungeeignet für sicherheitskritische Betriebsprozesse. Man muss dem Identity-Provider vertrauen.
3. Abgabe der Kontrolle an den jeweiligen Dienstanbieter.
4. Komplizierter zu implementieren. Die verwendeten Ressource-Server und Anwendungen müssen OAuth unterstützen.
5. Bei nicht planvoller Einführung/Umstellung oder unklarer Abgrenzung von lokalen oder “traditionellen” Logins können Verwirrung und Fehler seitens der Benutzer entstehen. Nicht zu unterschätzender Aufwand bei der Einführung/Umstellung.

5 WebAuthn: Programmierschnittstelle für eine direkte Authentifikation mittels Public Key-Verfahren

Dieser Versuch beschäftigt sich mit dem neuen Web-Authentifizierung-Standard `WebAuthn`*, welcher vom World Wide Web Consortium (W3C[1]) veröffentlicht wurde. Bei diesem Standard geht es um die Authentifikation von Benutzern mittels Public Key-Verfahren[2]. Ziel dieses Standards ist es, die Sicherheit des Authentifizierungsprozesses zu erhöhen, indem die passwortbasierte Authentifizierung ersetzt, beziehungsweise erweitert wird und gleichzeitig das Nutzererlebnis erhalten bleibt.*

Die Authentifizierung über Passwort ist in der Praxis immer noch die gängigste Methode. Dies, obwohl die Passwort-Authentifizierung einige Nachteile mit sich bringt. So zählen Passwörter wie beispielsweise `123456` *oder* `password` *erschreckenderweise nach wie vor zu den meist-genutzten Passwörtern überhaupt. Die Folgen daraus kann sich heute jeder selbst ausmalen. Mittels* `WebAuthn` *wird eine Passworteingabe abgelöst, weshalb auch die Möglichkeit eines Phishing-Angriffs dadurch eliminiert wird.*

Der WebAuthn-Standard umfasst folgende Parteien, siehe Abbildung 17:

1. User Agent (Browser)
2. Relying Party (Web Anwendung)
3. Authenticator (TPM-Chip, Yubikey, …)

`WebAuthn` wurde am 04. März 2019 veröffentlicht.

Sie werden im Folgenden nun die Authentifizierung mittels `WebAuthn` analysieren.

[1] Weitere Infos unter https://www.w3.org/TR/webauthn/

[2] Erklärung unter https://de.wikipedia.org/wiki/Public-Key-Authentifizierung

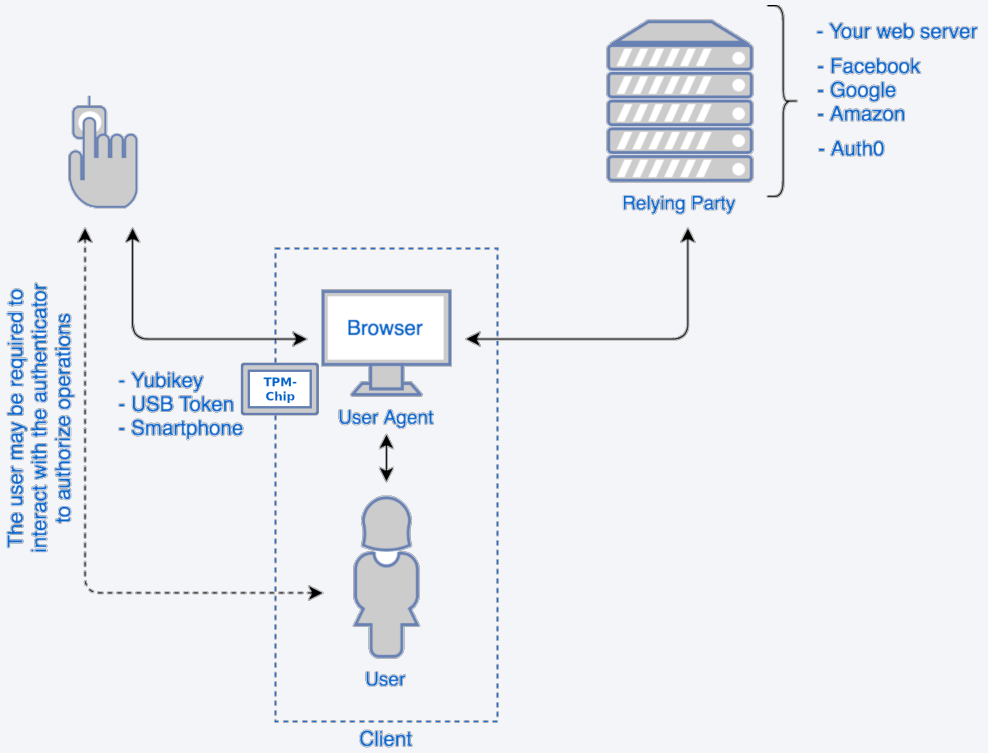


Abbildung 17: Übersicht über den Ablauf von `WebAuthn` mit Teilnehmern und beteiligten Geräten. (Quelle: https://auth0.com/blog/introduction-to-web-authentication/)

1. Ersetzt WebAuthn OAuth? Haben WebAuthn und OAuth dieselbe Funktionalität? Wenn nein, was sind die Unterschiede? Welchen Teil von OAuth ersetzt WebAuthn? Nehmen Sie sich Zeit für eine stichhaltige Antwort.

   Nein, WebAuthn und OAuth haben NICHT die selbe Funktionalität. WebAuthn kann als Addon für OAuth genutzt werden.

   - WebAuthn ist eine Browser-API, die die Authentifizierung von Benutzern *ohne* Passwörter handhabt. Anstelle von Passwörtern verwendet es Hardware-Authentifizierungsoptionen wie die integrierte Hardware eines Geräts wie den TPM-Chip oder einen Yubikey.
   - WebAuthn authentifiziert Benutzer, *und nicht mehr.* OAuth dagegen regelt viel mehr, vor allem auch den Zugriff auf eine API und weiteres. OAuth nutzt dafür aber gewöhnlicherweise einfache Passwörter. Hier ist keine Authentifizierungsstrategie vorgegeben bei OAuth. Diese Schwachstelle löst WebAuthn. WebAuthn ersetzt, wenn gewünscht, den Schritt in OAuth, in dem der Benutzer sein Passwort eingibt, da WebAuthn ein Ersatz für die Passwortauthentifizierung ist - nicht mehr und nicht weniger.
   - Aber: WebAuthn stellt einer App kein Zugriffstoken zur Verfügung, um API-Anforderungen zu stellen, da es nicht dafür konzipiert ist. Dies wird weiterhin mit dem etablierten Mechanismus mit OAuth gemacht.

5.1 Konfiguration von OWASP ZAP mit `Microsoft Edge`

Wie bereits in der Einleitung erwähnt, ist die Kompatibilität des Standards noch nicht vollumfänglich sichergestellt. Grundsätzlich können für WebAuthn verschiedene Authenticators herangezogen werden. Sie werden in diesem Versuch das TPM (Trusted Platform Module) als `platform` oder `Built In Authenticator` kennenlernen. TPM ist derzeit nur in Verbindung mit dem Webbrowser `Microsoft Edge` kompatibel, weshalb Sie in diesem Versuch diesen benutzen werden.

1. Starten Sie zuerst den Webbrowser **Microsoft Edge** und öffnen Sie die Webseite

   ```
   https://webauthn.io/
   ```

2. Starten Sie nun die Applikation `OWASP ZAP`, welche Sie schon aus dem vorgängigen Kapitel kennen.

   Speichern Sie die ZAP-Session wieder nicht.

3. Wie im vorherigen Versuch müssen Sie nun wieder das Zertifikat des ZAP-Proxys importieren.

   Sie haben im Versuch zu OAuth 2.0 das Root Certificate vom ZAP schon exportiert. Da der Browser Edge keinen eigenen Zertifikatsspeicher (wie Firefox) pflegt, sondern mit dem Windows Certificate Store arbeitet, muss das Zertifikat entsprechend dort importiert werden.

4. Klicken Sie auf `Start` und geben Sie im Startmenü den Befehl `certmgr.msc` ein, um den Zertifikatsmanager zu öffnen.

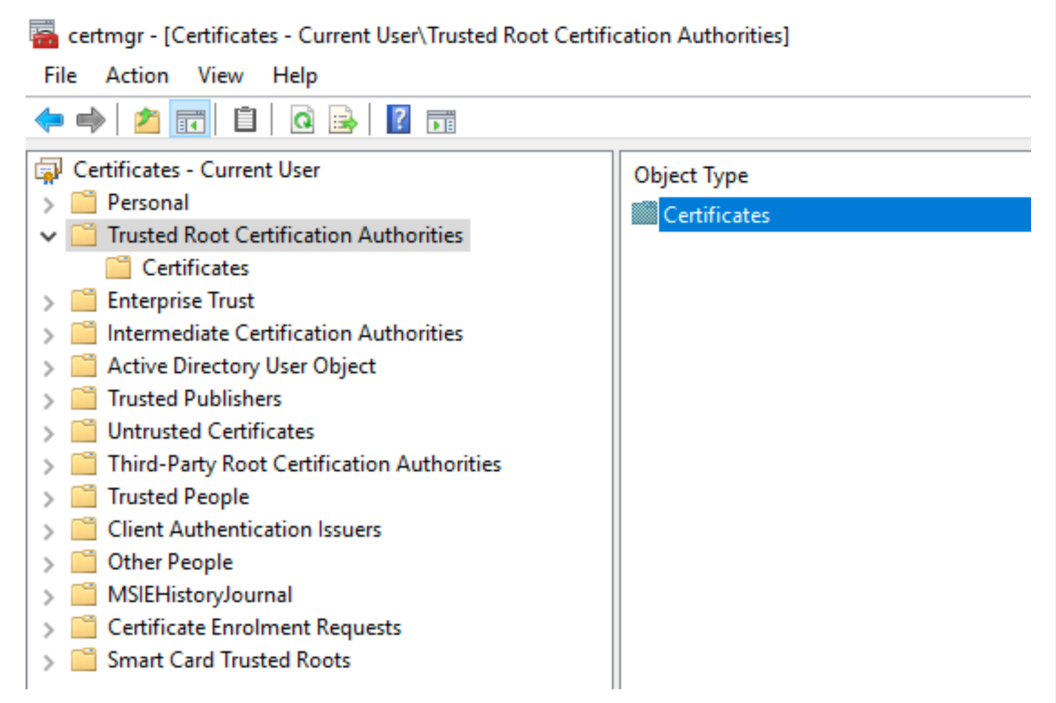


Abbildung 18: Screenshot des Windows Zertifikatsmanagers.

5. Nachdem sich der Zertifikatsmanager geöffnet ist, klicken Sie im Menü auf der linken Seite auf `Trusted Root Certification Authorities`.

6. Klicken Sie mit der rechten Maustaste auf den Eintrag `Certificates` und importieren Sie über `All tasks` das Zertifikat des OSWAP ZAP Proxys. Nachdem Sie nun das Zertifikat eingebunden haben, ist die Konfiguration des OWASP ZAP abgeschlossen. Im nächsten Schritt müssen Sie nun den Proxy in Microsoft Edge aktivieren.

7. Öffnen Sie die Einstellungen des `Microsoft Edge`-Browsers und klicken Sie dort auf den Menüpunkt `Edge` → `Settings` → `System and Performance` → `System` → `Open your computer's proxy settings`.

Im Gegensatz zur Proxy-Einstellung bei Webbrowsern wie Firefox oder Chrome nutzt Microsoft Edge die systemweiten Einstellungen von Microsoft Windows.

8. Sie sehen nun auf der rechten Seite `Manual proxy setup`.

9. Aktivieren Sie `Use a proxy server` und geben Sie wie bereits im vorherigen Versuch die Adresse des Proxys sowie den Port ein. Klicken Sie danach auf `Save`.

```
Adresse: 127.0.0.1
Port:    8080 (siehe Übung OAuth 2.0)
```

5.2 Konfiguration TPM & Windows Hello

Unter Trusted Platform Module (TPM)[1] wird eine Technologie verstanden, welche hardware-basierte Sicherheitsfunktionen zur Verfügung stellt. Dabei ist ein sogenannter TPM-Chip auf dem Motherboard des Rechners verbaut und führt kryptographische Operationen aus.

TPM sollte vorgängig auf Ihrem Rechner aktiviert worden sein. Kontrollieren Sie dies, indem Sie im Startmenü von Windows `tpm.msc` eingeben. Danach öffnet sich das Management das Trusted Platform Module (TPM).

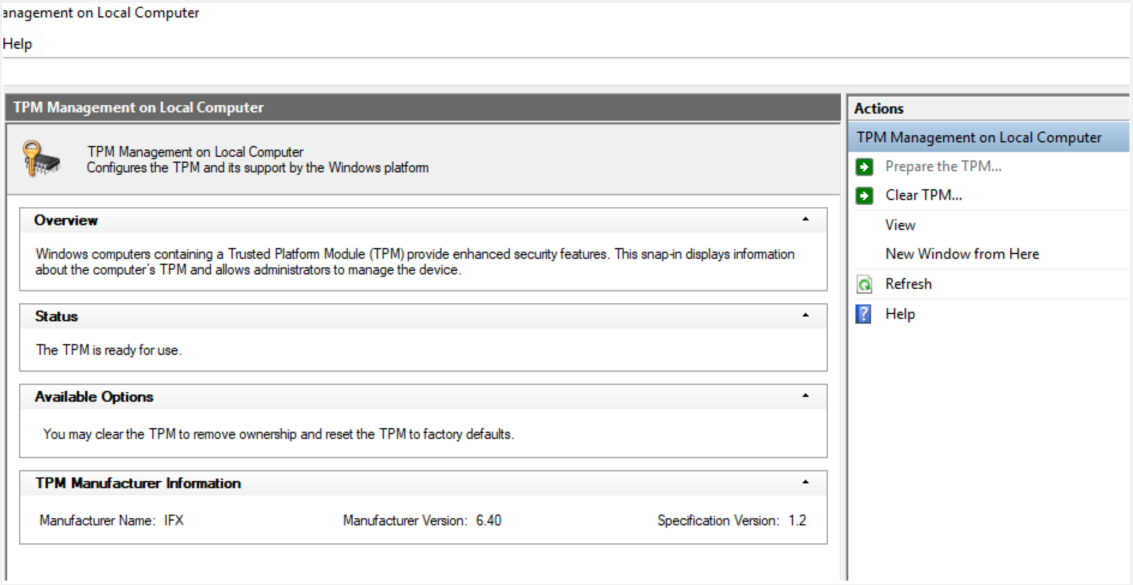


Abbildung 19: Screenshot der Trusted Platform Module Management-Konsole.

[1] Quelle: https://docs.microsoft.com/en-us/windows/security/information-protection/tpm/trusted-platform-module-overview

1. Kontrollieren Sie, ob der Status ihres TPM `ready for use` ist. Sollte dies nicht der Fall sein, sehen Sie auf der rechten Seite unter `Actions` den Punkt `Prepare the TPM`. Klicken Sie auf diesen Punkt und aktivieren Sie TPM. Ihr Rechner wird danach einen Neustart durchführen.

2. Damit wir mit dem TPM arbeiten können, muss im Vorfeld `Windows Hello` konfiguriert

werden.

3. Unter `Windows Hello`[1] werden alternative Authentifizierungsmethoden zur klassischen Passworteingabe zusammengefasst. Zu diesen angebotenen Methoden gehören beispielsweise Gesichtserkennung, Fingerabdruckscanner, Smart Cards oder PIN-Eingabe in Verbindung mit einem TPM-Chip auf dem PC. Auf dem Gerät, welches Sie benutzen, ist derzeit der TPM-Chip mit PIN-Eingabe als alternative Authentifizierungsmethode verfügbar.

[1] Quelle: https://www.golem.de/specials/windows_hello/

Genau genommen muss man zwischen (1) den eigentlichen Hardware-Elementen für die Generierung von Credentials und (2) den Zusatz-Authentifizierungselementen für die Steigerung der Usability unterscheiden.

- Gesichtserkennung, Fingerabdruckscanner oder PIN-Eingabe werden oft *nur* als Hilfmittel genutzt, um den TPM-Chip (oder eine Smartcard oder einen Yubikey) zu entsperren.
- Für die eigentliche Authentifizierung wird dann nur der TPM-Chip (oder der Yubikey) und seine kryptographische Operationen genutzt.

Der Ablauf ist wie folgt: 1) Entsperren des Hardware-Elements für die sichere Generierung der Credentials (gewöhnlicherweise TPM-Chips) via Gesichtserkennung, Fingerabdruck oder PIN-Eingabe, 2) Generierung von Credentials durch das Hardware-Element (TPM-Chip, Yubikey oder Smartcard), die für die Authentifizierung genutzt werden.

4. Öffnen Sie die Windows Settings → `Accounts` → `Sign-in options`. Fügen Sie einen `Windows Hello`-PIN hinzu und notieren Sie sich diesen. Sie benötigen den PIN später wieder! Wenn Sie zur Eingabe eines Benutzerpasswords aufgefordert werden, verwenden Sie den Benutzernamen `cisco` und das Passwort `cisco`.

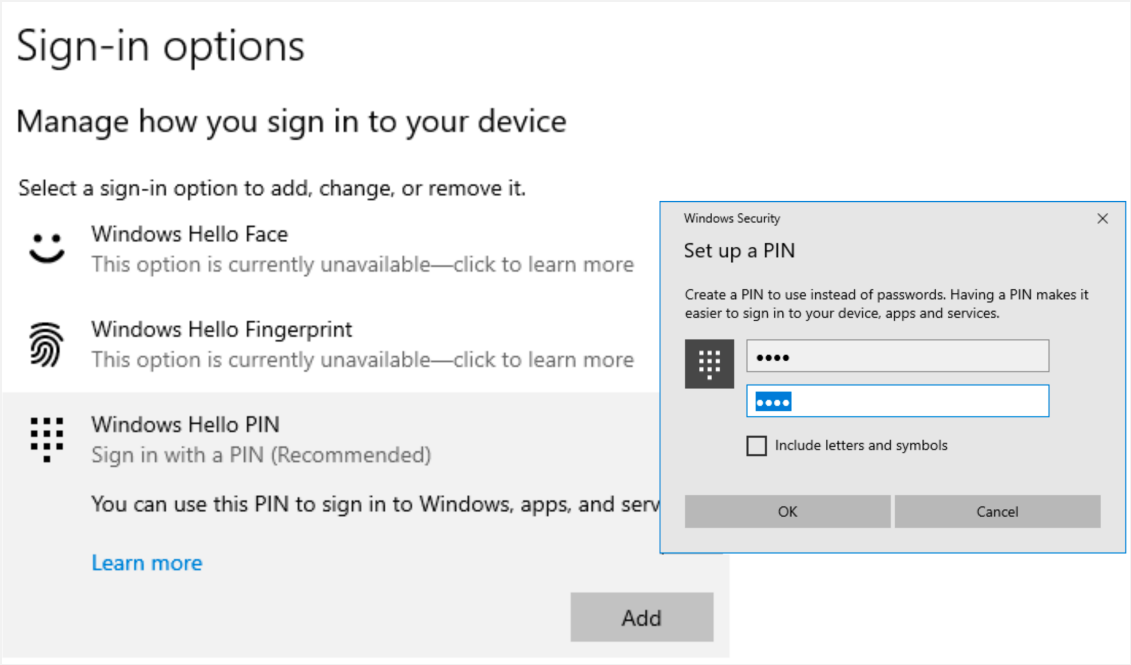


Abbildung 20: Die PIN-Eingabe bei `Windows Hello`.

Nachdem Sie die PIN-Eingabe als Authentifizierungsmethode gewählt haben, ist die Konfiguration des TPM-Moduls abgeschlossen.

5.3 Kennenlernen des WebAuthn-Mechanismus

Nachdem Sie nun alle Einstellungen und Konfigurationen vorgenommen haben, lernen Sie nun die theoretischen und konzeptionellen Hintergründe des Standards kennen.

Wie bereits erwähnt umfasst der WebAuthn-Standard drei Parteien: User Agent, Relying Party und den Authenticator. Hinter dem User Agent (= dem Webbrowser) steht immer ein Benutzer, welcher sich auf einer Webanwendung (beispielsweise einem Internethändler wie Amazon) einloggen will, um die Dienste des Webseitenbetreibers in Anspruch zu nehmen.

Bei der Anmeldung auf einer Website mit WebAuthn kommen immer zwei Prozesse ins Spiel:

1. die Registrierung des Benutzers (`Sign-Up`: Erstellen des Benutzeraccounts und Registrieren eines Benutzers bei der Webanwendung oder -seite) sowie
2. der eigentliche Login (nach der Registrierung). Diese zwei Prozesse werden Sie nun in den folgenden Abschnitten analysieren.

5.3.1 Sign Up: Erstellen eines Bentzerkontos bei der Webanwendung (Registrierung)

Wir beginnen mit dem ersten essentiellen Schritt des Prozesses: der Registrierung (quasi das Sign-Up bei der Webseite).

Hier wird der Benutzer zunächst mit der Webanwendung bekannt gemacht und registriert. Bei normaler, einfacher Passwort-Authentifizierung entspricht dies einer Registrierung, bei der dann das Passwort und der Benutzername gewählt werden. Auch bei WebAuthn muss der Nutzer, bevor er/sie sich überhaupt einloggen kann, bei dem Dienst registriert werden – also bekannt gemacht werden. Dies läuft über den TPM-Chip. Der Nutzer muss nahezu nichts machen, ausser den TPM-Chip entsperren und einen Benutzernamen wählen.

Zum Entsperren des TPM-Chips ist lediglich der Windows Hello-PIN nötig. Der PIN wird nur lokal an den TPM-Chip übertragen, dient zum Entsperren und wird nicht ins Internet gesendet.

Auch in Richtung der Website wird kein Passwort mehr genutzt. Als Ersatz für das Passwort wird vom TPM-Chip für den Benutzer ein Schlüsselpaar (Private Key und Public Key) generiert. Ein Benutzername (z. B. E-Mail-Adresse) und die Angabe des gewünschten Authenticators, der die Authentifizierung anstelle des Nutzers mit seinem Passwort durchführt (in unserem Beispiel der TPM-Chip), reichen also für die Registrierung von Seiten des Nutzers aus. Der Registrationsprozess mittels WebAuthn ist in Abbildung 22 abgebildet.

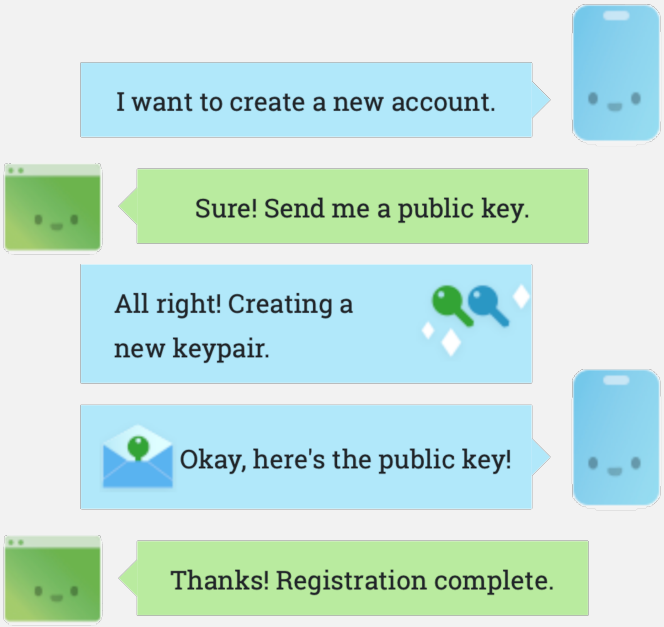


Abbildung 21: Ablauf des Sign Ups eines Benutzers (Quelle: https://webauthn.guide)

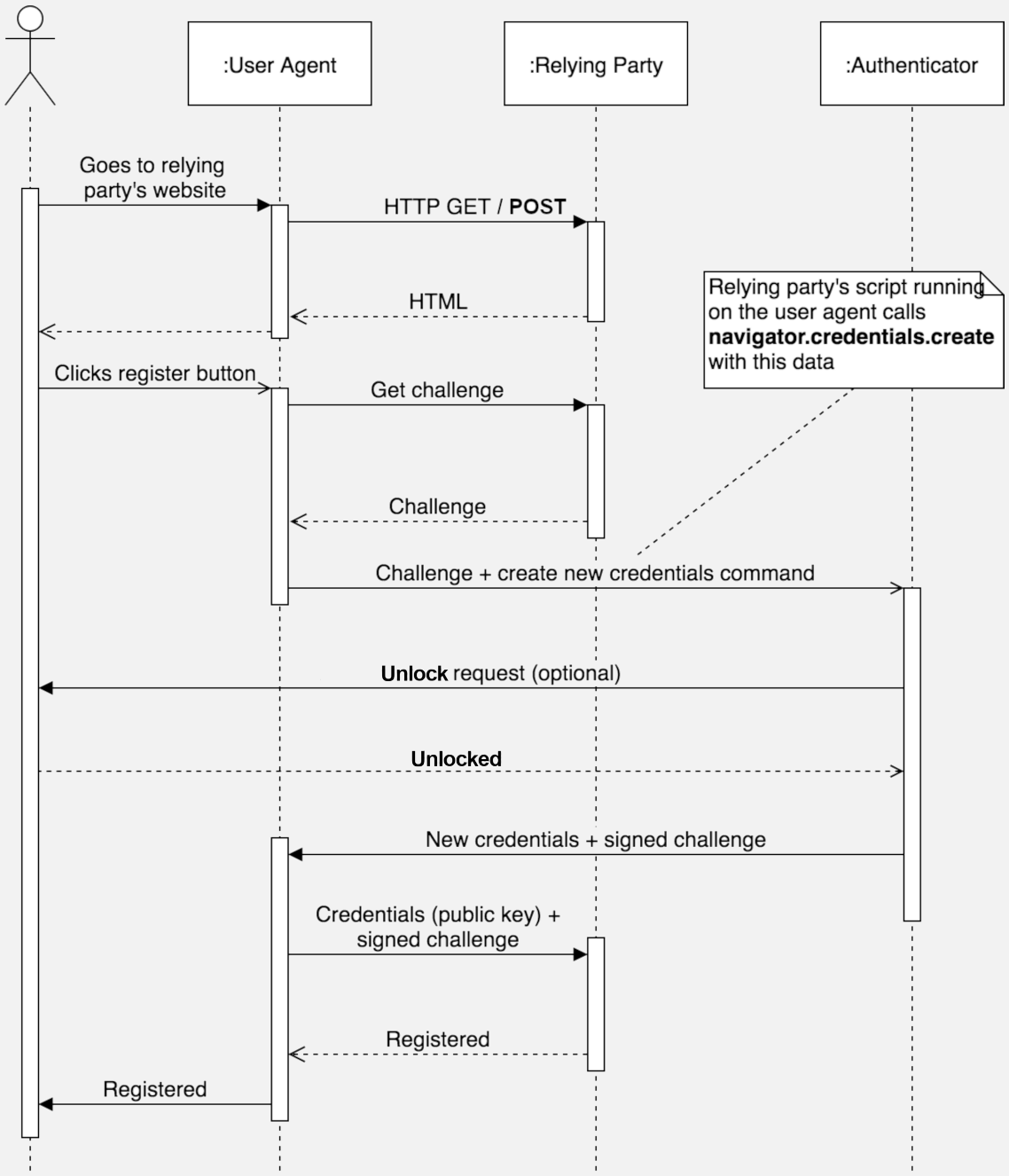


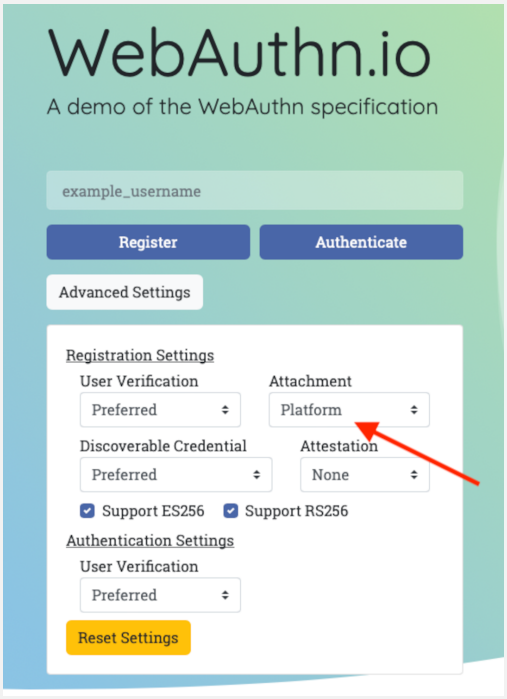
Abbildung 22: Registrierungsprozess bei WebAuthn. (Quelle: https://auth0.com/blog/introduction-to-web-authentication/)

1. Kehren Sie zurück zu OWASP ZAP und starten Sie, wie im vorherigen Versuch, eine neue ZAP-Proxy Session (Icon `weisses Blatt` → `New Session`), um mit der Analyse zu starten.

> Werden unnötige Seiten geladen, können Sie, wie im vorherigen Versuch, alle unnötigen Seiten von der Analyse ausschliessen (Rechtsklick auf die unerwünschten Seiten → `Exclude from`). Sie sollten dann wesentlich weniger geladene Seiten haben. Dies wird Ihnen die Analyse erleichtern. Sie können diesen Vorgang natürlich für weitere aufkommende Seiten wiederholen.

2. Wechseln Sie nun zum Webbrowser zurück und geben Sie einen gewünschten Benutzernamen, beziehungsweise eine E-Mail, ein.

3. Wechseln Sie in den Einstellungen `Advanced Settings` bei dem Parameter `Attachement` zu `Platform`.



Bei den Authenticators unter `Attachment` kann zwischen `Cross-Platform` und `Platform` unterschieden werden. Bei `Cross-Platform` handelt es sich um alle externen Authenticators (wie z.B. Smartphone, USB-Keys, etc.).

4. Nachdem Sie alle Eingaben getroffen haben, klicken Sie auf `Register` bzw. `Registrieren`. **GEBEN SIE DEN PIN NOCH NICHT EIN!**

5. Wechseln Sie zuerst zurück in den OWASP-Proxy und suchen Sie nun den Request, welcher durch das Klicken auf den `Register`-Button bzw. `Registrieren`-Button verursacht worden ist.

6. Öffnen Sie den Request, indem Sie doppelt auf ihn klicken.

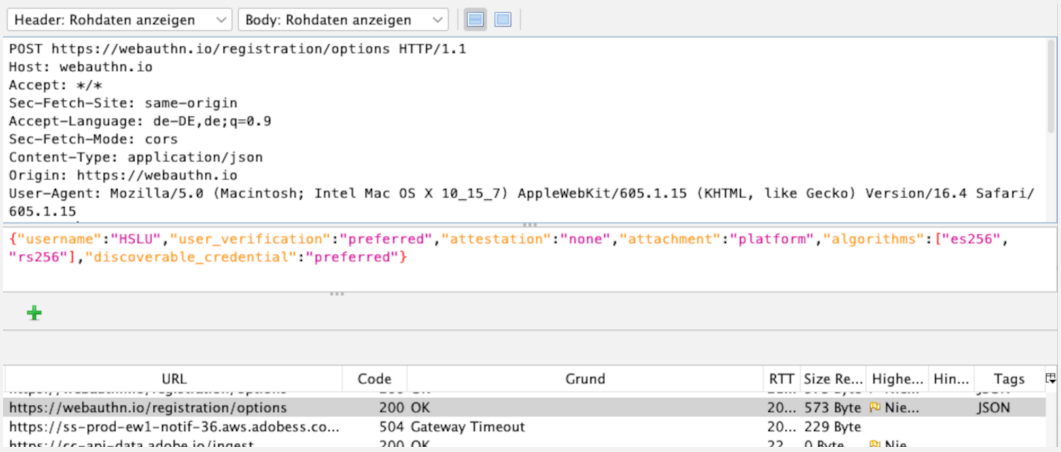


Abbildung 23: HTTP-POST-Request der Registrierungsanfrage.

7. Sehen Sie sich den Request genauer an. Was können Sie aus diesem ableiten? Googlen Sie gegebenfalls die verschiednen Parameter, die gesendet werden.

Hier beginnt der eigentliche WebAuthn-`Sign-Up`-Prozess - die Registrierung des Benutzers bei der Webseite. *Wichtig:* Der wichtigste Schritt hierbei ist eigentlich nur *das Senden des Public Keys zum Server*.

Es wird aber nicht nur der Public Key (und der Benutzername) übertragen, sondern es werden auch Einstellungen, *wie* sich der Nutzer später ausweisen wird, gesetzt und übertragen. Weiter werden Verschlüsselungsalgorithmen für das Public Key-Verfahren vorgeschlagen, die der Authenticator (= unser TPM-Chip) unterstützt, hier `ES256` und `RS256`.

- *username:* Dies ist der Name des Benutzers, der sich gerade registriert. Die Webseite verwendet diesen Name, um dem Benutzer einen Public Key zuzuordnen. Es wird eigentlich empfohlen, keine personenbezogenen Daten als Name zu verwenden, da diese möglicherweise in der Webseite gespeichert werden.

- *attachment:* Diese Einstellung hilft der Webseite, weitere Einschränkungen für die Art der für die Registrierung zulässigen Authentifikatoren vorzunehmen. In unserem Beispiel geben wir vor, dass wir uns mit Hilfe des "normalen" Plattform-Authentifikators (auf den Labor-PCs `Windows Hello` (sonst oft auch Apple `Touch ID`) authentifizieren möchten. Wir könnten auch beispielsweise angeben, dass wir einen plattformübergreifenden Authentifikator (`Cross-Platform`) (wie ein Smartphone) anstelle des "normalen" Plattform-Authentifikators wie `Windows Hello` oder `Touch ID` registrieren möchten.
- *attestation:* `attestation` bedeutet *Beglaubigung* auf Deutsch. Die Beglaubigungsdaten sind ***die*** *zentralen Daten*, die der TPM-Chip bei der Authentifizierung oder Registrierung an die Webseite sendet. Mit diesen Daten kann die Webseite beim Login dann später verifizieren, dass dieser Login tatsächlich von dem nun registrierten Nutzer stammt. Generell ist das mindestens der Public Key des Nutzers und eine Challenge, die der Server zuvor bereitgestellt und generiert hat, die vom TPM-Chip des Nutzer signiert wurde.

  Hier, bei der Registrierung, können nun aber Optionen in Bezug auf das `attestation`-Objekt eingestellt werden: Clients können angeben, wie die Beglaubigungsdaten an den Server weitergegeben werden.

  - Der Wert "none" gibt an, dass man die Angaben des TPM-Chips mindestens teilweise ignoriert (oft bis auf die eigentliche Challenge und Public Key). Grundsätzlich kann der Nutzer damit Daten im `attestation`-Objekt beim Erstellungsvorgang entfernen und Just-in-Time-Funktionen zum Generieren und Signieren von Zertifikaten von einem anderen Soft-Token nutzen, das er definiert.
  - Der Wert "indirekt" bedeutet, dass der Client anonymisierte `attestation`-Objekte präferiert (keine Informationen über den Erstellvorgang eines Keys im `attestation`-Objekt, beispielsweise im Public Key).
  - "direct" bedeutet, dass die Webseite die Beglaubigungsdaten vom TPM-Chip direkt nutzen soll und diese weitergeben soll.
- *algorithms:* Dies ist ein Array von Objekten, die beschreiben, welche öffentlichen Schlüsseltypen für den Client für die Beglaubigungsdaten akzeptabel sind, siehe Public Key-Verfahren, hier `ES256` und `RS256`.

8. Sehen Sie sich nun die Response genauer an. Vergleichen Sie insbesondere die Parameter des Requests mit den Angaben in der Response.

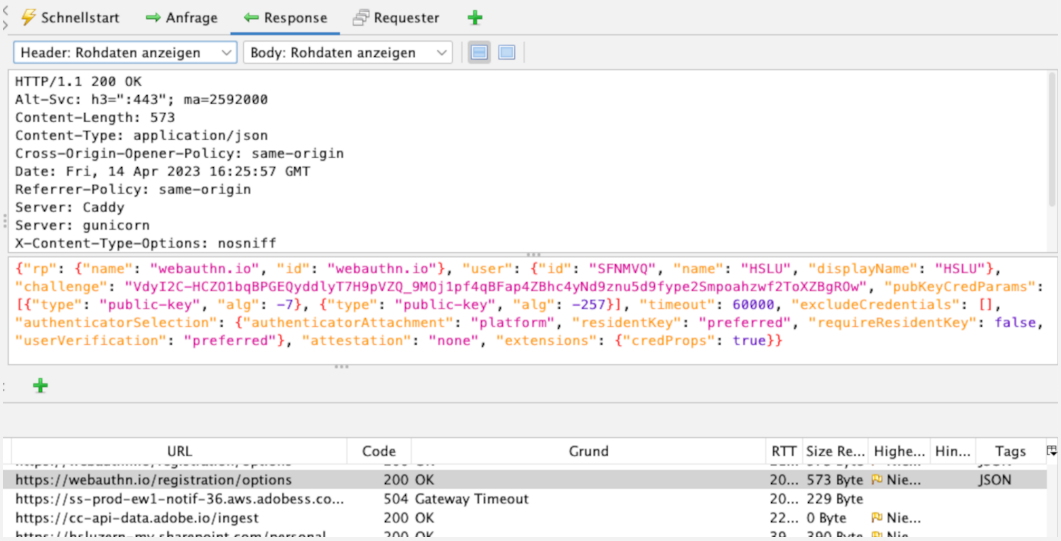


Abbildung 24: HTTP-POST-Response der Registrierungsanfrage.

Vergleichen Sie hierzu das Flussdiagramm. Wo befinden Sie sich derzeit?

Wir befinden uns nun direkt nach dem Schritt `Challenge`. Wir bekommen vom Server eine Challenge auf unsere Antwort zurück. Ebenfalls werden mögliche Einstellungen und Informationen vom Webserver in einem JSON an uns zurückgesendet, insbesondere sind das die ID des neu registrierten Nutzers und sein Anzeigename. Nun muss die Challenge von unserem Browser gelöst - sprich mit dem Private Key signiert - werden.

9. Wozu wird die Challenge benötigt, die in der Response (siehe Abbildung 24) gesendet wird?
   Hinweis: Challenge-Response-Authentifizierung.

   Die `Challenge` wird in der Regel benötigt, um einen Nutzer zu authentifizieren (Authentifizierung), *ohne dabei ein Passwort zu übermitteln oder immer dasselbe Passwort verwenden zu müssen.* Dadurch werden insbesondere "Replay-Attacken" verhindert. Bei diesem Prozess generiert der Server eine Challenge, die aus zufälligen Zeichen besteht. Der Client signiert diese Challenge mit einem privaten Schlüssel und sendet sie über HTTP zurück an den Server. Der Server überprüft die Signatur mithilfe des dazugehörigen öffentlichen Schlüssels. Abschliessend erhält der Client eine HTTP-Antwort (`HTTP Response`), die darüber informiert, ob die Challenge akzeptiert oder abgelehnt wurde.

   Hier bei der Registierung wird die Challenge genutzt, um einen "Test-Login" durchzuführen. Im Prinzip muss hier beim Schritt "Registrierung" eigentlich nur der `Public Key` an den Server gesendet werden - die Challenge ist eigentlich nicht unbedingt nötig. Dass eine Challenge zusätzlich (wie beim normalen Login-Vorgang) gelöst wird, dient dazu den Public-Key zu "testen". Wie beim Login wird überprüft, ob der Public Key zu dieser gelösten Challenge passt. Es wird quasi ein Dry-Run durchgeführt, ob der Public Key auch wirklich in Verbindung mit einer signierten Challenge funktioniert.

10. Wofür steht das Kürzel `rp`, das in der Response gesendet wird? Weshalb ist dieser `rp`-Eintrag in der Response wichtig?

    rp: Steht für „relying party" (vertrauende Partei); In unserem Fall ist dies die Webseite, wo wir uns gerade registrieren, sprich https://webauthn.io. Diese Webseite vertraut den Daten des Clients und authentifiziert später gemäss dieser Daten den Benutzer. Diese `rp` muss eine Teilmenge der Domäne der Webseite sein, die derzeit im Browser geöffnet und genutzt wird, sonst meldet der Browser einen Fehler. Eine gültige `rp` für diese Seite ist beispielsweise `webauthn.io`.

Die `pubKeyCredParams` in der Response (siehe Abbildung 24) geben an, welche Methode beziehungsweise Typ und welche Algorithmen für die Erstellung der Credentials gewählt werden sollen und von der Web Application akzeptiert werden. Die einzelnen Zahlen beim Parameter `alg` stehen dabei für die verschiedenen Algorithmen. So steht -7 beispielsweise für den Algorithmus `ES256`. Die Reihenfolge der Einträge ist geordnet von `most preferred` zu `least preferred`. Der Client geht dabei den «best-effort»-Weg und wählt die für ihn passende Methode aus, um sich zu authentifzieren.

- Weitere Informationen hierzu erhalten Sie unter https://www.w3.org/TR/webauthn/#dom-publickeycredentialcreationoptions-pubkeycredparams.
- Informationen zu den einzelnen Algorithmen erhalten Sie hier: https://www.iana.org/assignments/cose/cose.xhtml#algorithms.

11. Wechseln Sie wieder zum Webbrowser und geben Sie im nächsten Schritt nun Ihren vorgängig gewählten PIN-Code ein. Hier kann es sein, dass der Authenticator ein Timeout zurückgibt, da der PIN nicht innerhalb einer Minute eingegeben wurde. Dieses Fenster können Sie schliessen und danach können Sie sich erneut einloggen.

12. Welcher Stelle, beziehungsweise welchem Schritt, im Flussdiagramm würden Sie die PIN-Eingabe zuordnen?

    Schritt: Unlock request (optional) (Entspricht der Authorisierung durch den Benutzer, dass der TPM genutzt werden darf und soll.)

13. Sehen Sie sich nun die Veränderungen nach der PIN-Eingabe im ZAP-Proxy an.

14. Sehen Sie sich noch einmal das Flussdiagramm des Registrierungsprozesses an. Wo befinden Sie sich jetzt? Wie weit lief der Vorgang nach Eingabe des PINs selbstständig weiter? Was ist alles passiert?

    Wir befinden uns am Ende des Flussdiagramms bei `Registered`. Dies schliessen wir daraus, dass wir die Meldung auf der Webseite erhalten haben, dass wir registriert sind. Die Webseite hat nach der PIN-Eingabe vom TPM-Chip den Public Key und die signierte Challenge erhalten und per HTTP-GET an die Webseite gesendet. In der HTTP-Response haben wir als Antwort darauf die Nachricht “Created” zurückbekommen.

15. Im letzten Schritt der Registrierung wurde das `attestationObject` vom TPM-Chip an den Webbrowser gesendet. Dieser hat das Objekt an https://webauthn.io weitergereicht. Das `attestationObject` ist das zentrale Rückgabeobjekt, das bei der Registrierung an die Webseite gesendet wird. Mit dem Versenden des Objekts ist die Registrierung auf Seiten des Clients vollzogen. Was wird unter dem `attestationObject` verstanden?
Hinweis: Sehen Sie sich das Flussdiagramm und den zugehörigen Schritt an.

Siehe nächsten Kasten unten für die Lösung.

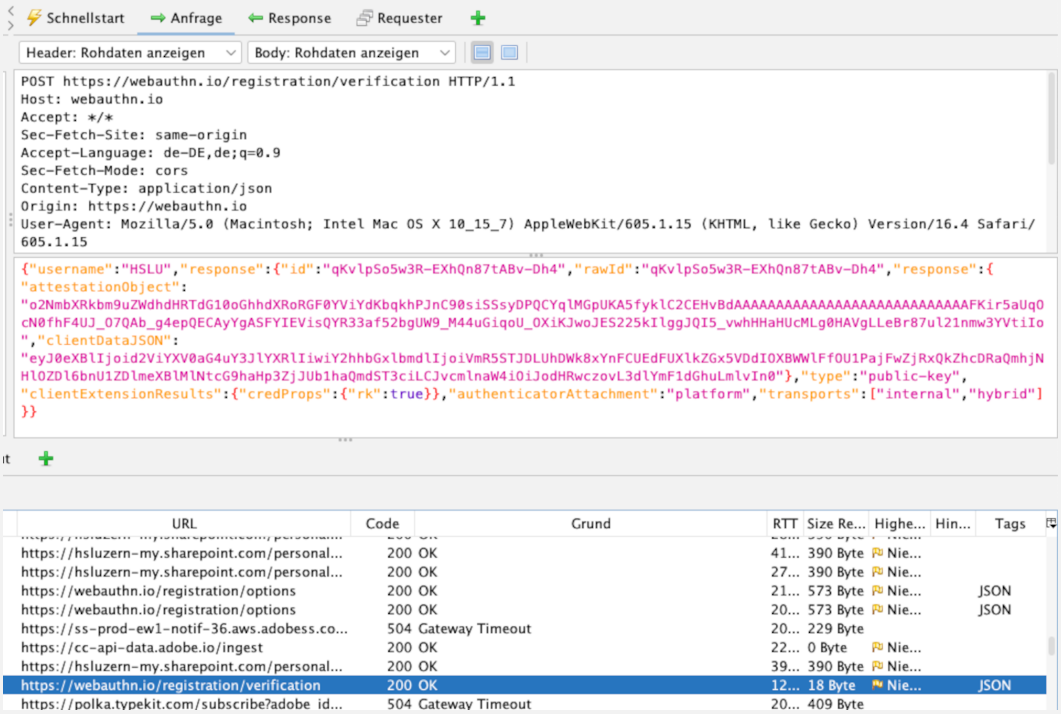


Abbildung 25: Das `attestationObject` wird bei der Registrierung als das zentrale Rückgabeobjekt an die Webseite gesendet.

Dies ist das zentrale Objekt der Registrierung. Authentifizierer (wie der TPM-Chip) MÜSSEN irgendeine Form der Beglaubigung bereitstellen, damit ein Nutzer authentifiziert werden kann. `attestation` bedeutet *Beglaubigung* auf Deutsch. Die Beglaubigungsdaten sind die Daten, die der TPM-Chip bereitstellt, sodass der Nutzer später "wiedererkannt" und zweifelsfrei authentifiziert werden kann. Mit den Daten im `attestationObject` kann die Webseite beim Login später eben verifizieren, dass dieser Login tatsächlich von dem nun registrierten Nutzer stammt.

In unserem Fall erstellt der TPM-Chip ein *Schlüsselpaar* bestehend aus privatem und öffentlichem Schlüssel. Der öffentliche Schlüssel wird zur Webseite im `attestationObject` geschickt und dient als Beglaubigungsobjekt für den Loginvorgang. Signiert der TPM-Chip nun eine Challenge beim Loginvorgang mit dem privaten Schlüssel und schickt er diese signierte Challenge zum Webserver zurück, so kann der Webserver mit dem bei der Registrierung erhaltenen öffentlichen Schlüssel nachprüfen, ob die Signatur eindeutig vom TPM-Chip des Benutzers gemacht wurde, der auch bei der Registrierung den öffentlichen Schlüssel geschickt hatte.

Der Authentifizierer (TPM-Chip) muss pro registriertem Benutzer (pro Anmeldeinformation) eine Beglaubigung erstellen, die von der WebAuthn-Relying-Party zum Verifizieren genutzt werden kann (das heisst die Relying-Party kann damit verfizieren, dass die Challenge korrekt gelöst wurde).

Typischerweise enthält das gesamte `attestationObject` neben dem öffentlichen Schlüssel noch zusätzlich eine schon signierte Challenge. Mit dieser signierten Challenge kann die Webseite initial testweise überprüfen, ob der gesendete öffentliche Schlüssel funktioniert und die Signatur bestätigt werden kann → Validieren der Beglaubigungsdaten (= öffentlicher Schlüssel). Weiter enthält das gesamte `attestationObject` typischerweise noch ein Zertifikat oder ähnliche Daten, die Herkunftsinformationen für den öffentlichen Beglaubigungsschlüssel bereitstellen, was es der vertrauenden Seite (relying party) ermöglicht, eine Vertrauensentscheidung zu treffen.

5.3.2 Login-Prozess

Nachdem Sie sich nun erfolgreich registriert haben, werden Sie jetzt den Login-Prozess analysieren.

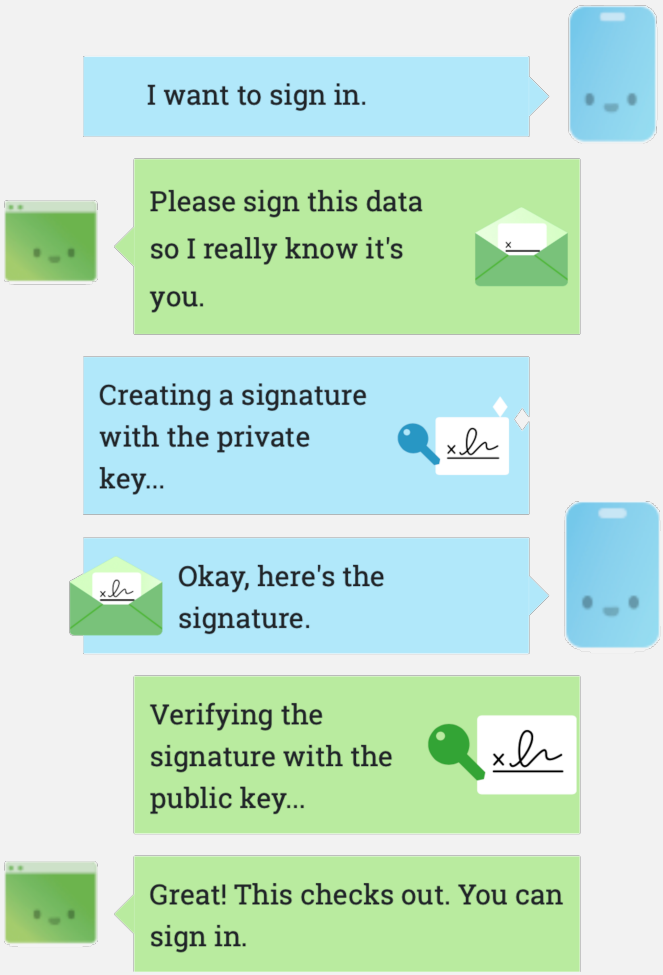


Abbildung 26: Ablauf des Logins. (Quelle: https://webauthn.guide)

Das Flussdiagramm des Login-Prozess sieht folgendermassen aus:

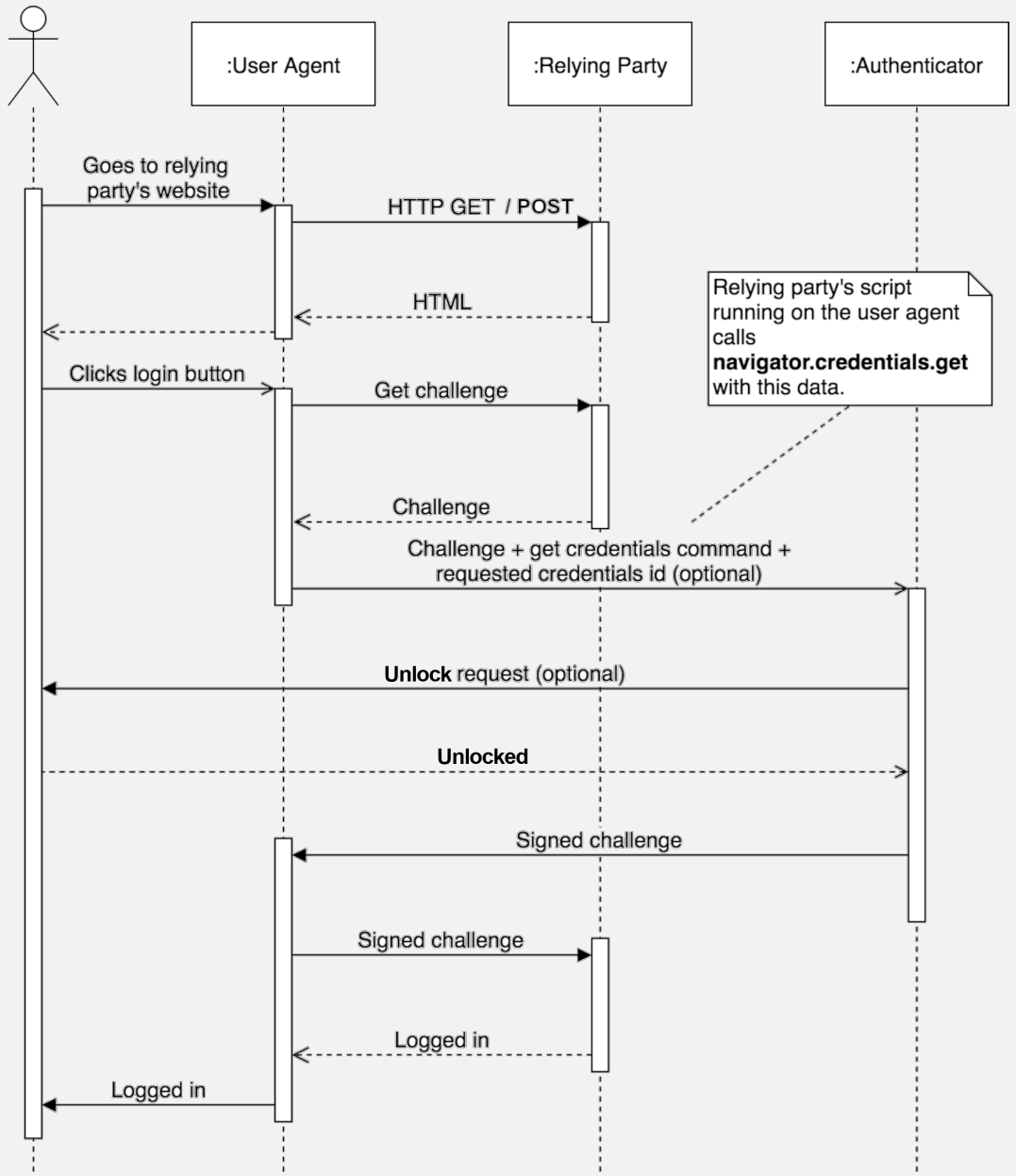


Abbildung 27: Login-Prozess WebAuthn. (Quelle: https://auth0.com/blog/introduction-to-web-authentication/)

1. Klicken Sie nun auf den `Authenticate`-Button, um sich auf der Webseite https://webauthn.io einzuloggen.
2. **Bevor Sie den PIN eingeben**, wechseln Sie nochmals in den ZAP-Proxy und sehen Sie sich die neuen Einträge an.
    1. HTTP-Request an https://webauthn.io/authentication/options
    2. HTTP-Antwort zum Request zu https://webauthn.io/authentication/options

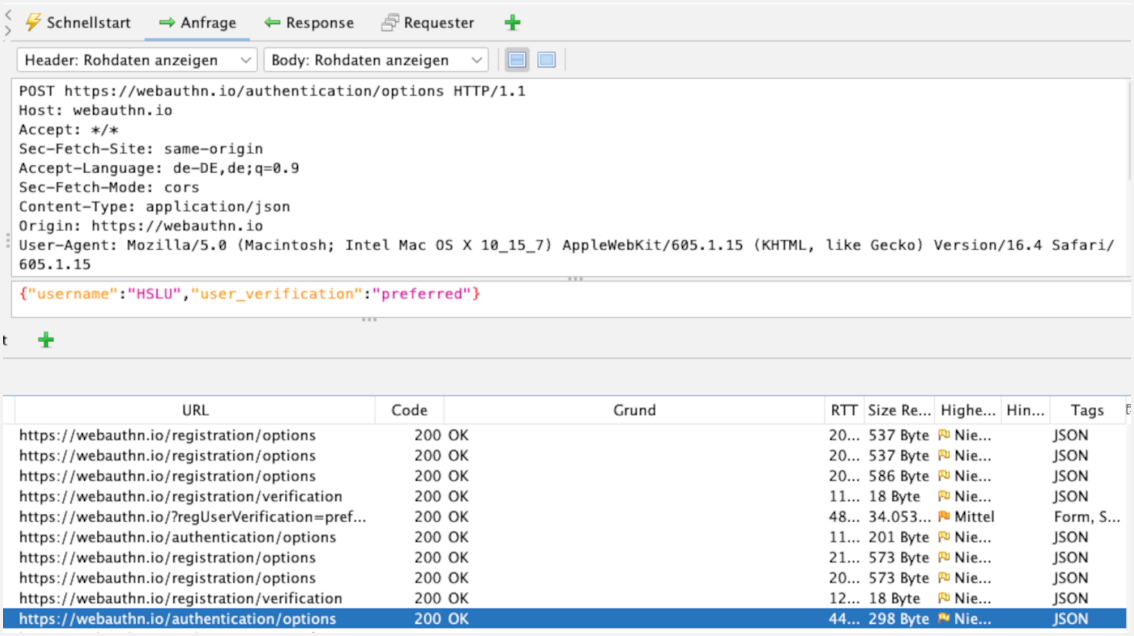


Abbildung 28: *Anfrage:* HTTP-Request des Login-Vorgangs zur Übermittelung des Benutzernamens.

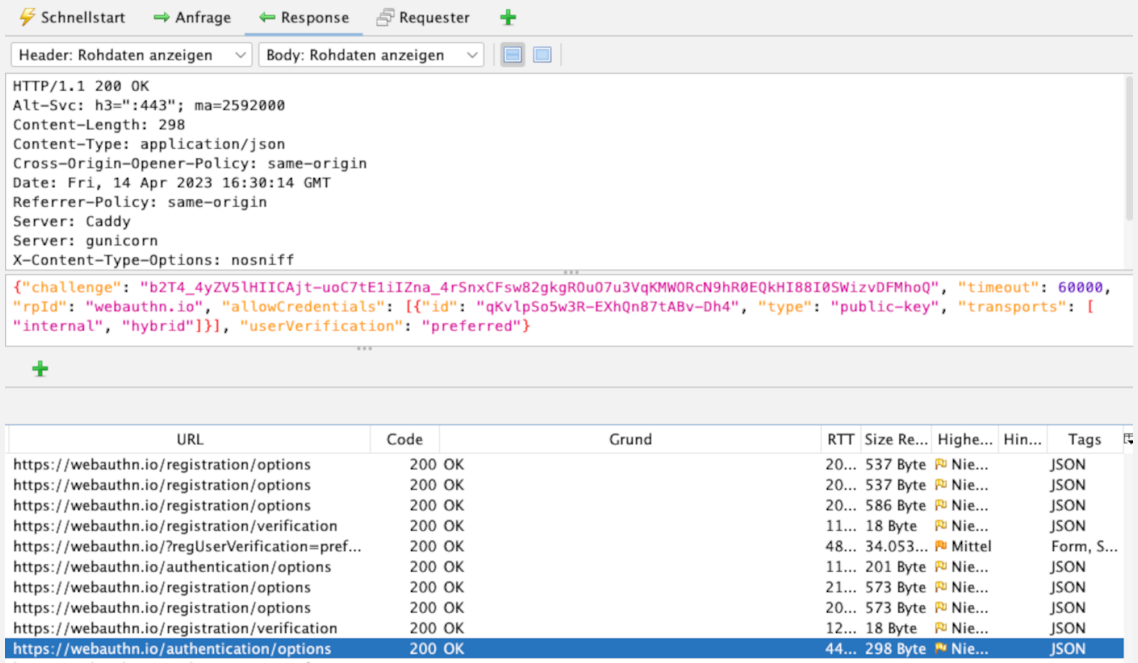


Abbildung 29: *Antwort:* HTTP-Response (Antwort) von der Webseite zum Benutzer mit der zu signierenden Challenge.

3. Sie sehen hier eine zufällige ID im Zweig `allowCredentials`. Handelt es sich bei dieser ID um die gleiche wie bei der Registrierung? Verifizieren Sie dies im Traffic.

   Ja, es ist die gleiche ID, siehe Abbildung 25 (Dort wurden die Registrierung abgeschlossen und die registrierten Credentials bestätigt.).

4. Wie gerade gesehen: Die wichtigste Aufgabe des *ersten* Kommunikationsschritts beim Login ist das Senden des Benutzernamens zur Webseite und das Zurücksenden der Challenge vom Webserver zum Client.

   Hier wird, wie bei der Registrierung, wieder eine Challenge zum Client gesendet. Erklären Sie die Aufgabe der Challenge in diesem Fall im Vergleich zur Challenge bei der Registrierung.

   Hier wird die Challenge zum Client gesendet, damit dieser die Challenge signiert und der Nutzer authentifiziert werden kann. Später wird dann die Signatur mit Hilfe des bei der Registrierung erhaltenen öffentlichen Schlüssels überprüft, ob die Signatur eindeutig vom TPM-Chip des Nutzers stammt, der die Registrierung gemacht hatte. Dies entspricht dem Normalfall: Ein Nutzer wird authentifiziert, das heisst die Identität des Nutzers am Browser wird dadurch bestätigt, sprich der Benutzer am Browser ist der gleiche Benutzer wie bei der Registrierung.
   → Wurde die Signatur eindeutig mit dem privaten Schlüssel erstellt, der zum bei der Registrierung erhaltenen öffentlichen Schlüssel passt, dann ist der Nutzer bestätigt und eindeutig authentifiziert.

   Im Gegensatz hierzu ist das Signieren der Challenge *bei der Registrierung* nur ein Test mit dem die Webseite initial testweise überprüfen kann, ob der öffentliche Schlüssel, der gerade bei der Registierung mitgeliefert wurde, überhaupt funktioniert → Validieren der Registrierungsdaten. Technisch ist der Vorgang der gleiche, der Zweck ist aber ein grundlegend anderer:

   - Registrierung → Validieren der Registrierungsdaten: initiales Überprüfen, ob der öffentliche Schlüssel in Ordnung ist und funktioniert.
   - Login → Authentifizierung des Nutzers.

5. Wechseln Sie nun wieder zum Webbrowser und geben Sie den PIN ein. Nach der PIN-Eingabe haben Sie sich erfolgreich eingeloggt (siehe Abbildung 30).

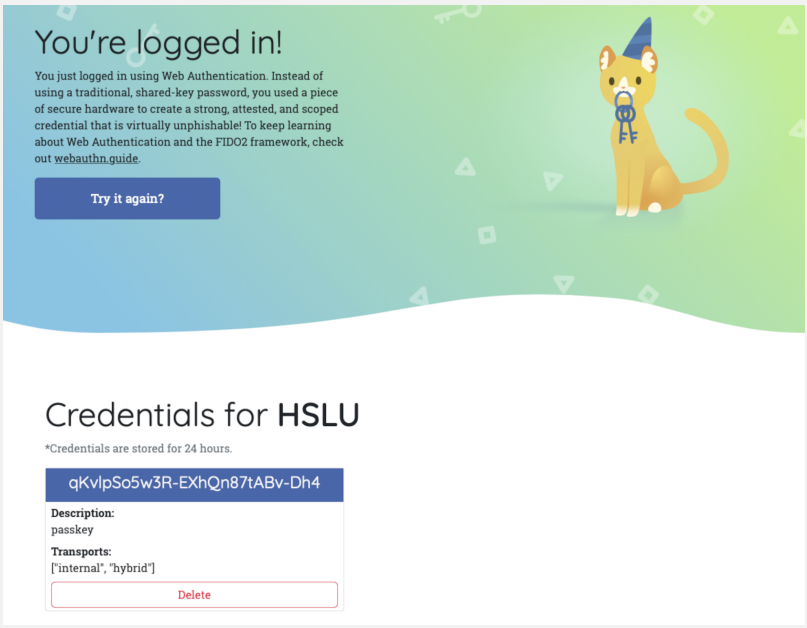


Abbildung 30: Meldung, dass der Benutzer erfolgreich angemeldet werden konnte, das heisst, die Authentifizierung des Benutzers war erfolgreich.

6. Schauen Sie sich nun zum Schluss die Anfrage und die Antwort des letzten Schritts an:
    1. das Senden der *signierten* Challenge vom Client zurück zur Webseite per HTTP-Request für den Login und
    2. die Antwort von der Website, die bestätigt, dass die signierte Challenge unter Verwendung des öffentlichen Schlüssels, der vom Client während der Registrierung an die Website gesendet wurde, verifiziert werden konnte.

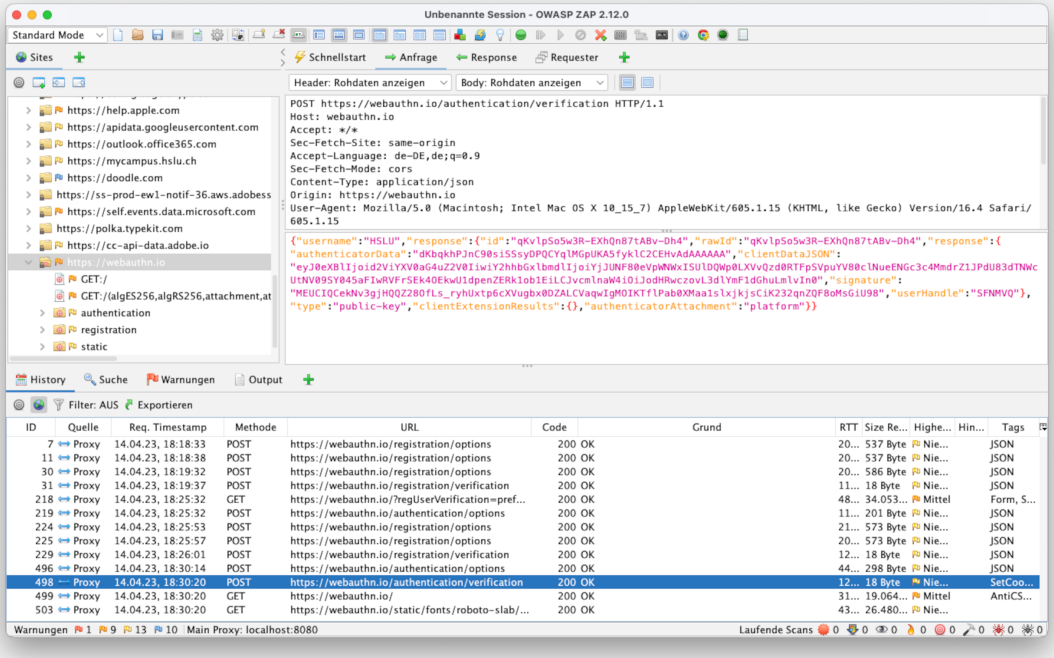


Abbildung 31: HTTP-Request zum Übertragen der signierten Challenge zur Webseite.

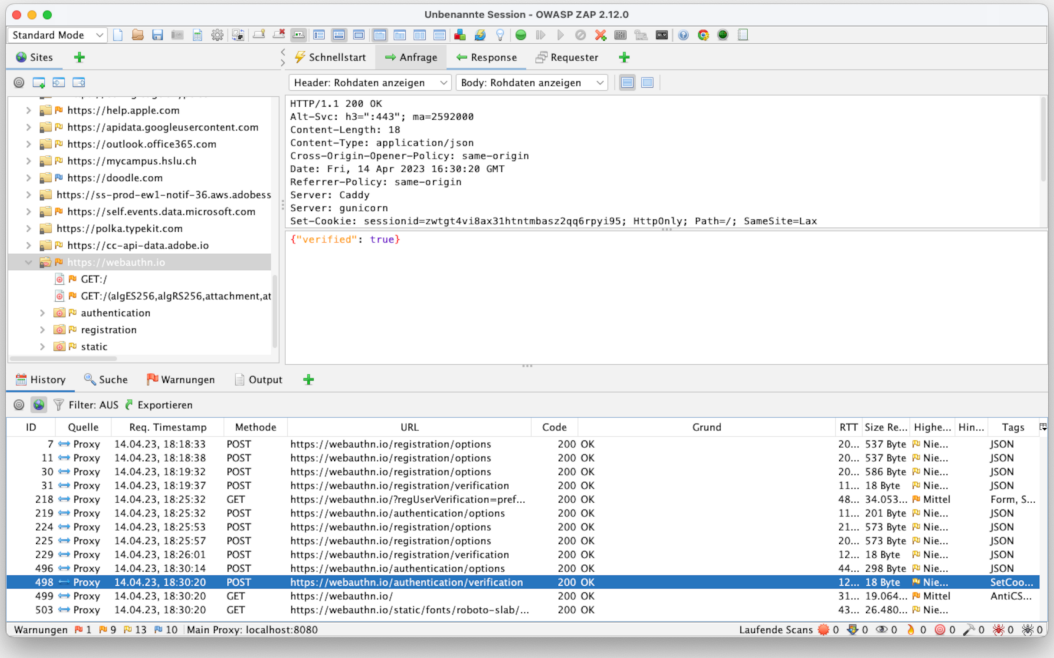


Abbildung 32: Antwort der Webseite, dass die Signatur mit dem bei der Registierung erhaltenen öffentlichen Schlüssel verifiziert werden konnte.

5.4 Abschluss WebAuthn

Analog zur Vorteile/Nachteile-Tabelle im ersten und zweiten Teil dieser Übung sollen Sie dies nun für WebAuthn festhalten. Nehmen Sie auch Bezug zu den folgenden Themen: Security, Portierbarkeit, Integration, Login Experience.
Vorteile:

- *Sicherheit:* Keine Passwörter mehr, sondern Schlüssel zur Authentifizierung. Das Verwenden von Passwörtern ist nur bedingt sicher: Entweder sie lassen sich knacken (mit Brute Force oder Rainbow Tables beispielsweise) oder die Kennwörter werden per Phishing ausgespäht. Bei WebAuthn gibt es keine Passwörter, die ein Anwender unbedacht wählen oder weitergeben kann.
- *Sicherheit:* Da der Login-Prozess lokal an den Rechner gebunden ist (TPM), kann man sich nur von diesem einen Rechner einloggen.
- *Sicherheit:* Durch die Nutzung einer signierten Challenge werden immer andere Daten für jeden Authentifizierungsvorgang an den Webserver übertragen. → Schutz vor “Replay-Attacken”.
- *Sicherheit:* Gleiches gilt auch für Man-in-the-middle-Attacken: Da bei dem neuen Standard keine Identitätsdaten über das Internet übertragen werden, bringt auch ein Man-in-the-Middle-Angriff, bei dem die Daten während der Übertragung abgegriffen werden könnten, nichts. Durch das Public-Key-Verfahren ist zudem die Übermittlung des Public Keys kryptografisch gesichert.
- *Sicherheit:* Es werden Technologien wie TPM verwendet, um Schlüsselpaare pro Webseite zu erzeugen und sicher zu verwahren.
- *Sicherheit:* Dass alle sensiblen Daten im TPM-Chip des Nutzers bleiben, ist gleichzeitig der Vorteil für Webseiten-Betreiber. Anbietern von Diensten, die eine Anmeldung voraussetzen, müssen derzeit sehr viel Energie und Kompetenz in die Sicherung von Passwörtern und Benutzernamen stecken. Es ist fatal, wenn es Kriminellen gelingt, in die Datenbanken des Anbieters einzudringen. Auf Unternehmen, die einen solchen Angriff nicht abwenden können, kommen schwerwiegende Konsequenzen zu, und auch die leidtragenden Nutzer müssen mit erheblichem Datenmissbrauch rechnen - besonders dann, wenn sie die Anmeldedaten auch auf anderen Plattformen verwenden.
- *Integration:* Der ganze Prozess ist sehr gut ins Windows- oder Apple-Betriebssystem integriert. Der Benutzer hat die Möglichkeit sich mit einem einzigen PIN oder Fingerabdruck, welchen er auch beim OS verwendet, bei Webdiensten einzuloggen.

- *Login Experience:* Besonders der Komfort dürfte Anwender überzeugen. Es müssen keine Informationen mehr auswendig gelernt werden. Das Verwenden von Passwörtern ist schliesslich nicht mehr notwendig.
- *Login Experience:* Hinzu kommt: Zweifaktor-Authentifizierung (2FA) oder Multifaktor-Authentifizierung (MFA) ist ein relativ zeitaufwendiger Vorgang. WebAuthn funktioniert schneller und ist somit nutzerfreundlicher. WebAuthn gilt überdies als sicherer als die Multi-Faktor-Authentisierung. Zwar bietet das zusätzliche Identitätsmerkmal, das beim Anmelden per MFA abgefragt wird, einen zusätzlichen Schutz, dies ist aber dennoch nicht komplett ohne Risiko. Manche Authentisierungsmerkmale - wie ein One-Time-Password per SMS - können relativ leicht abgefangen werden. Ausserdem sind auch diese Kurzzeitpasswörter zu beliebten Angriffspunkten von Phishing geworden.

Nachteile:

- *Integration:* Unterstützung ist bei diversen oder alten oder neuen Webbrowsern und Applikationen noch nicht gegeben.
- *Integration:* Bei manchen Betriebssystemen ist dieser Prozess (noch) nicht integriert.
- *Login Experience:* Man kann sich nur genau mit einem Rechner/PC, der den TPM-Chip, der bei der Registierung genutzt wurde, enthält, bei einer Webseite anmelden, siehe auch Vorteile “Sicherheit”.
- Falls der Rechner kaputt geht oder die gespeicherten Credentials im TPM-Chip verloren gehen, kann man sich nicht mehr einloggen/authentifizieren. Beispiel: Ein neuer Authenticator soll für ein bestehendes Konto angemeldet werden. Wenn beispielsweise das Hardware-Token verloren geht, braucht man ein neues. Dieses neue Token lässt sich dann nicht ohne weiteres mit dem bestehenden Profil verknüpfen - das wäre ein zu großes Sicherheitsrisiko. Stattdessen muss entweder ein Ersatz-Authenticator vorhanden sein, der genau für diesen Einsatz bestimmt ist, oder man muss einen Reset vornehmen. Letzteres entspricht etwa dem Zurücksetzen eines Passworts und ist vor allem für Dienste geeignet, die keinen hohen Sicherheitsstandard verlangen.
- Etwas kompliziert zu implementieren

6 Aufräumen

Zu guter Letzt müssen Sie noch ein paar wenige Aufräumschritte unternehmen, damit das System für die nächsten Laborbenutzer wieder wie gewohnt funktioniert.

1. Entfernen Sie den gesetzten PIN wieder.
2. Wechseln Sie in die Firefox Einstellungen `Settings` zu `General` → `Network Settings` und wählen Sie `No proxy`.

Im nächsten Schritt räumen wir den Proxy in Microsoft Edge auf.

3. Öffnen Sie wieder die Einstellungen des `Microsoft Edge`-Browsers und klicken Sie dort auf den Menüpunkt `Settings`. Sie sehen nun auf der linken Seite `System`. Klicken Sie auf `Open your computer's proxy settings`.
4. Deaktivieren Sie unter `Manual proxy setup` die Option `Use a proxy server`.

Gratulation, somit ist die Übung abgeschlossen.

ENDE

Laden Sie das ausgefüllte PDF mit Hilfe des vorbereiteten Abgabeordners in ILIAS hoch.